

[illegible]

Giáo viên hướng dẫn
(Ký tên và ghi rõ họ tên)

[illegible]

Vĩnh Long, ngày tháng năm
Thành viên hội đồng
(Ký tên và ghi rõ họ tên)

LỜI CẢM ƠN

Để hoàn thành đồ án này, em đã nhận được rất nhiều sự quan tâm, động viên và giúp đỡ từ quý Thầy Cô, gia đình và bạn bè.

Trước hết, em xin gửi lời cảm ơn chân thành và sâu sắc nhất đến Tiến sĩ Nguyễn Bảo Ân – Giảng viên hướng dẫn. Thầy là người đã tận tâm chỉ dạy, định hướng và đồng hành cùng em trong suốt quá trình thực hiện. Những lời khuyên chuyên môn và sự khích lệ của Thầy đã giúp em vượt qua nhiều khó khăn để hoàn thiện đồ án một cách tốt nhất.

Em xin trân trọng cảm ơn Ban Giám hiệu cùng quý Thầy Cô Trường Đại học Trà Vinh đã tạo điều kiện thuận lợi, truyền đạt kiến thức và kinh nghiệm quý báu cho em trong suốt thời gian học tập tại trường.

Em cũng xin gửi lời cảm ơn đến gia đình – chỗ dựa vững chắc luôn ủng hộ em trong mọi hoàn cảnh, cùng tất cả bạn bè đã luôn sẻ chia, hỗ trợ em trong quá trình học tập và thực hiện đề tài.

Mặc dù đã rất cố gắng, đồ án chắc chắn không tránh khỏi những thiếu sót. Em rất mong nhận được sự góp ý từ quý Thầy Cô để bài làm được hoàn thiện hơn.

Em xin chân thành cảm ơn!

Vĩnh Long, tháng 12 năm 2025

Sinh viên thực hiện

Nguyễn Thanh Hiếu

MỤC LỤC

CHƯƠNG 1 TỔNG QUAN	12
1.1. Bối cảnh và bài toán hỏi đáp pháp luật	12
1.2. Các hướng tiếp cận và hạn chế.....	12
1.3. Yêu cầu và tiêu chí của hệ thống đề xuất.....	12
1.4. Phạm vi triển khai và giới hạn	13
1.5. Kết luận chương	13
CHƯƠNG 2 NGHIÊN CỨU LÝ THUYẾT	14
2.1. Tổng quan RAG.....	14
2.2. Embedding và truy hồi ngữ nghĩa.....	15
2.2.1. Từ Word Embeddings đến Sentence Embeddings	15
2.2.2. Sentence-BERT cho Semantic Search	16
2.2.3. Embedding Models cho Tiếng Việt	17
2.2.4. Similarity Metrics và Retrieval Process.....	18
2.3. Vector database với PostgreSQL + pgvector.....	19
2.3.1. Vector database và Approximate Nearest Neighbor Search.....	19
2.3.2. FAISS – Facebook AI Similarity Search	19
2.3.3. pgvector - Vector Extension cho PostgreSQL	20
2.3.4. Indexing Strategies trong pgvector	20
2.3.5. So sánh FAISS vs pgvector	21
2.4. Reranking với Cross-Encoder.....	22
2.4.1. Bối cảnh two-stage retrieval	22
2.4.2. Cơ chế hoạt động của Cross-Encoder	22
2.4.3. So sánh Bi-Encoder và Cross-Encoder	22
2.4.4. BGE Reranker cho bài toán đa ngữ	23
2.5. Context Expansion và Citation Tracing.....	23
2.6. LLM và chiến lược sinh câu trả lời.....	24
2.6.1. Tổng quan về Large Language Models	24
2.6.2. In-Context Learning và Few-Shot Prompting	25
2.7. Session Management & Conversational RAG.....	26
2.7.1. Từ Single-turn đến Multi-turn Conversation	26
2.7.2. Session State và Quản lý Ngữ cảnh Hội thoại	26
2.7.3. Context Pinning, xử lý Follow-up Questions và Contextualization	26
2.8. Tối ưu hóa cho tiếng Việt	27
2.8.1. Thách thức ngôn ngữ đối với truy hồi ngữ nghĩa	27
2.8.2. Tokenization, word segmentation và lựa chọn mô hình	27
2.8.3. Lưu ý tiền xử lý cho văn bản pháp luật tiếng Việt.....	27

2.9. Kiến trúc Microservices.....	28
2.9.1. Từ Monolithic đến Microservices.....	28
2.9.2. Các đặc trưng chính của Microservices và áp dụng vào hệ thống.....	28
2.10. Object Storage (MinIO).....	29
2.11. Phương pháp đánh giá	29
2.12. Kết luận chương.....	30
CHƯƠNG 3 HIỆN THỰC HÓA NGHIÊN CỨU.....	32
3.1. Phân tích yêu cầu	32
3.1.1. Actor và use-case	32
3.1.2. Yêu cầu chức năng.....	34
3.1.3. Yêu cầu phi chức năng.....	35
3.2. Thiết kế kiến trúc tổng thể	36
3.2.1. Kiến trúc mức cao (High-level architecture)	36
3.2.2. Thiết kế module theo microservices	38
3.2.3. Cấu hình mạng và Port Mapping	39
3.2.4. Luồng giao tiếp	40
3.2.5. Thiết kế API & giao tiếp liên service	43
3.3. Thiết kế dữ liệu	44
3.3.1. Mô hình mức quan niệm.....	45
3.3.2. Mô hình mức luận lý.....	46
3.3.3. Mô hình mức vật lý.....	47
3.4. Thiết kế pipeline RAG.....	48
3.4.1. Document ingestion flow (upload → Knowledge Base)	48
3.4.2. Admin flow	49
3.4.3. Query flow	49
3.4.4. Cơ chế cải thiện hội thoại	51
3.4.5. Chính sách sinh câu trả lời.....	52
3.4.6. Pipeline điền biểu mẫu & quét CCCD.....	52
3.5. Thiết kế giao diện	54
3.5.1. UI/UX user chat	54
3.5.2. Admin dashboard.....	55
3.5.3. Form fill page.....	56
3.6. Triển khai và vận hành.....	57
3.7. Kết luận chương.....	57
CHƯƠNG 4 KẾT QUẢ NGHIÊN CỨU.....	58
4.1. Môi trường thực nghiệm	58
4.1.1. Cấu hình phần cứng và phần mềm.....	58
4.1.2. Dữ liệu và tập câu hỏi đánh giá	58
4.2. Kết quả chức năng	60

4.2.1. Chức năng hỏi đáp pháp luật	60
4.2.2. Cơ chế Clarification (làm rõ truy vấn).....	61
4.2.3. Conversational RAG (Document Pinning)	62
4.3. Đánh giá chất lượng câu trả lời.....	63
4.3.1. Chiến lược Document-centric Retrieval	63
4.3.2. Các chỉ số đánh giá	64
4.3.3. Kết quả đánh giá ở mức đoạn văn.....	64
4.3.4. Kết quả đánh giá ở mức văn bản.....	65
4.3.5. Đánh giá chất lượng câu trả lời.....	66
4.3.6. Phân tích kết quả theo lĩnh vực.....	66
4.4. Đánh giá hiệu năng	67
4.4.1. Thời gian phản hồi.....	67
4.4.2. Phân tích thời gian xử lý theo từng bước.....	68
4.4.3. Đánh giá mức độ đáp ứng yêu cầu phi chức năng.....	68
CHƯƠNG 5 KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN.....	69
5.1. Kết luận.....	69
5.2. Hướng phát triển	69
5.2.1. Mở rộng dữ liệu	69
5.2.2. Cải thiện mô hình.....	70
5.2.3. Triển khai thực tế.....	70
5.2.4. Tối ưu hiệu năng và vận hành.....	70
DANH MỤC TÀI LIỆU THAM KHẢO.....	71
PHỤ LỤC	72

DANH MỤC HÌNH ẢNH

Hình 2.1. Quy trình hoạt động của mô hình RAG	15
Hình 3.1. Sơ đồ use-case	33
Hình 3.2. Sơ đồ kiến trúc tổng quan	37
Hình 3.3. User Flow – UC01	41
Hình 3.4. Admin Flow – UC08.....	42
Hình 3.5. Mô hình mức quan niệm	45
Hình 3.6. Mô hình mức luận lý	46
Hình 3.7. Mô hình mức vật lý	47
Hình 3.8. Document ingestion flow (Upload → Knowledge Base)	48
Hình 3.9. Luồng xử lý Query flow.....	50
Hình 3.10. Vòng đời phiên hội thoại trong hệ thống	51
Hình 3.11. Pipeline hỗ trợ điền biểu mẫu hành chính.....	53
Hình 3.12. Giao diện trang hỏi đáp pháp luật	54
Hình 3.13. Giao diện hiển thị kết quả trả lời và nguồn tham chiếu	55
Hình 3.14. Giao diện Admin Dashboard.....	55
Hình 3.15. Giao diện điền biểu mẫu trước khi thực hiện quét CCCD	56
Hình 3.16. Giao diện sau khi hệ thống quét CCCD thành công và điền form.....	56
Hình 4.1. Giao diện nhập truy vấn hỏi đáp pháp luật	60
Hình 4.2. Kết quả trả lời và văn bản tham chiếu.....	61
Hình 4.3. Cơ chế yêu cầu làm rõ truy vấn.....	61
Hình 4.4. Kết quả trả lời sau khi làm rõ truy vấn.....	62
Hình 4.5. Lựa chọn và ghim văn bản pháp luật	62
Hình 4.6. Trả lời dựa trên văn bản đã ghim	63

DANH MỤC BẢNG BIỂU

Bảng 2.1. So sánh FAISS vs pgvector.....	21
Bảng 3.1. Use-case cho Người dùng (User).....	34
Bảng 3.2. Use-case cho Quản trị viên (Admin)	35
Bảng 3.3. Yêu cầu phi chức năng.....	36
Bảng 3.4. Phân rã các microservices trong hệ thống LegalRAG	38
Bảng 3.5. Phân nhóm service theo cấu hình port và phạm vi truy cập	39
Bảng 4.1. Thông số hệ thống.....	58
Bảng 4.2. Thống kê dữ liệu văn bản.....	59
Bảng 4.3. Phân bố câu hỏi test theo lĩnh vực	59
Bảng 4.4. Kết quả đánh giá ở mức đoạn văn.....	65
Bảng 4.5. Kết quả đánh giá ở mức văn bản.....	65
Bảng 4.6. Chất lượng câu trả lời của hệ thống	66
Bảng 4.7. Kết quả đánh giá theo lĩnh vực	67
Bảng 4.8. Kết quả đo thời gian phản hồi của hệ thống	67
Bảng 4.9. Phân tích thời gian xử lý theo từng bước.....	68

DANH MỤC TỪ VIẾT TẮT

Từ viết tắt	Tên đầy đủ	Diễn giải
AI	Artificial Intelligence	Trí tuệ nhân tạo
NLP	Natural Language Processing	Xử lý ngôn ngữ tự nhiên
LLM	Large Language Model	Mô hình ngôn ngữ lớn
LLMs	Large Language Models	Các mô hình ngôn ngữ lớn
RAG	Retrieval-Augmented Generation	Sinh văn bản có tăng cường truy hồi
LegalRAG	Legal Retrieval-Augmented Generation	Hệ thống RAG cho lĩnh vực pháp luật
ANN	Approximate Nearest Neighbor	Tìm kiếm láng giềng gần đúng
FAISS	Facebook AI Similarity Search	Thư viện tìm kiếm vector hiệu năng cao
BERT	Bidirectional Encoder Representations from Transformers	Mô hình ngôn ngữ hai chiều
SBERT	Sentence-BERT	Mô hình embedding ở mức câu
BGE	Beijing General Embedding	Dòng mô hình embedding / reranking
API	Application Programming Interface	Giao diện lập trình ứng dụng
REST	Representational State Transfer	Kiểu thiết kế API dựa trên HTTP
HTTP	HyperText Transfer Protocol	Giao thức truyền thông web
SDK	Software Development Kit	Bộ công cụ phát triển phần mềm
S3	Simple Storage Service	Chuẩn lưu trữ đối tượng
SQL	Structured Query Language	Ngôn ngữ truy vấn cơ sở dữ liệu
ACID	Atomicity, Consistency, Isolation, Durability	Thuộc tính giao dịch CSDL
JWT	JSON Web Token	Cơ chế xác thực dựa trên token
OCR	Optical Character Recognition	Nhận dạng ký tự quang học
CCCD	Căn cước công dân	Giấy tờ định danh cá nhân
GPU	Graphics Processing Unit	Bộ xử lý đồ họa
CPU	Central Processing Unit	Bộ xử lý trung tâm
P90	90th Percentile	90% truy vấn không vượt ngưỡng
P95	95th Percentile	95% truy vấn không vượt ngưỡng
Corpus	Text Corpus	Tập văn bản dùng cho xử lý ngôn ngữ

TÓM TẮT ĐỒ ÁN CHUYÊN NGÀNH

Trong bối cảnh nhu cầu tra cứu thủ tục hành chính và thông tin pháp luật tại Việt Nam ngày càng gia tăng, việc ứng dụng các mô hình ngôn ngữ lớn (LLM) vào bài toán hỏi đáp pháp luật đặt ra nhiều thách thức liên quan đến độ chính xác và khả năng kiểm chứng thông tin. Đồ án này nghiên cứu và xây dựng hệ thống hỏi đáp pháp luật tiếng Việt dựa trên kỹ thuật Retrieval-Augmented Generation (RAG), nhằm khắc phục hạn chế của các mô hình sinh ngôn ngữ thuần túy bằng cách ràng buộc câu trả lời vào các văn bản pháp luật chính thống.

Hệ thống được thiết kế theo kiến trúc microservices, tách biệt các thành phần xử lý chính như sinh embedding ngữ nghĩa, truy hồi văn bản, tái xếp hạng kết quả và sinh câu trả lời. Dữ liệu pháp luật được quản lý dưới dạng tài liệu gốc kết hợp với vector embedding trong cơ sở dữ liệu PostgreSQL tích hợp pgvector, cho phép truy hồi ngữ nghĩa hiệu quả. Đồ án áp dụng chiến lược Document-centric Retrieval nhằm đảm bảo mỗi câu trả lời được tổng hợp từ một văn bản pháp luật thống nhất, đồng thời tích hợp cơ chế hội thoại nhiều lượt, document pinning và yêu cầu làm rõ truy vấn để hạn chế sai lệch ngữ cảnh.

Kết quả thực nghiệm trên tập dữ liệu thủ tục hành chính cho thấy hệ thống đạt hiệu quả cao trong việc xác định đúng văn bản pháp luật liên quan và cung cấp câu trả lời có căn cứ, với thời gian phản hồi phù hợp cho các ứng dụng tương tác. Bên cạnh chức năng hỏi đáp, hệ thống còn hỗ trợ pipeline điền biểu mẫu và quét căn cước công dân, cho thấy tiềm năng ứng dụng thực tiễn trong lĩnh vực hành chính công..

MỞ ĐẦU

Trong những năm gần đây, việc ứng dụng trí tuệ nhân tạo vào lĩnh vực hành chính công và pháp luật ngày càng được quan tâm, đặc biệt là các hệ thống hỗ trợ người dân tra cứu thông tin và thực hiện thủ tục hành chính. Tuy nhiên, phương thức tra cứu truyền thống dựa trên tìm kiếm từ khóa hoặc đọc thủ công các văn bản pháp luật thường gây khó khăn cho người sử dụng phổ thông do nội dung phức tạp, thuật ngữ chuyên ngành và cấu trúc văn bản dài.

Sự phát triển của các mô hình ngôn ngữ lớn (Large Language Models – LLMs) mở ra khả năng xây dựng các hệ thống hỏi đáp pháp luật bằng ngôn ngữ tự nhiên. Dù vậy, việc sử dụng LLM một cách độc lập trong lĩnh vực pháp luật tiềm ẩn nhiều rủi ro, đặc biệt là hiện tượng sinh thông tin không chính xác hoặc thiếu căn cứ pháp lý, làm giảm độ tin cậy của hệ thống.

Trước những hạn chế đó, kỹ thuật Retrieval-Augmented Generation (RAG) được đề xuất như một hướng tiếp cận phù hợp nhằm kết hợp khả năng sinh ngôn ngữ của mô hình ngôn ngữ lớn với cơ chế truy hồi thông tin từ các nguồn pháp luật chính thống. Bằng cách gắn chặt quá trình sinh câu trả lời với các văn bản được truy hồi, RAG góp phần nâng cao độ chính xác, tính kiểm chứng và khả năng truy vết nguồn thông tin.

Xuất phát từ yêu cầu thực tiễn nêu trên, đề án này tập trung nghiên cứu, thiết kế và hiện thực hóa một hệ thống hỏi đáp pháp luật tiếng Việt dựa trên kỹ thuật Retrieval-Augmented Generation, hướng đến hỗ trợ người dân tra cứu thủ tục hành chính một cách thuận tiện, chính xác và minh bạch. Nội dung đề án lần lượt trình bày cơ sở lý thuyết, kiến trúc hệ thống, quy trình triển khai, kết quả thực nghiệm và các hướng phát triển trong tương lai..

CHƯƠNG 1 TỔNG QUAN

1.1. Bối cảnh và bài toán hỏi đáp pháp luật

Nhu cầu tìm hiểu pháp luật là nhu cầu thiết yếu của xã hội. Tuy nhiên, hệ thống văn bản pháp luật hiện nay có số lượng khổng lồ, chồng chéo và liên tục cập nhật/thay thế. Người dùng phổ thông thường bị "ngợp" giữa ma trận kết quả tìm kiếm, không xác định được đâu là văn bản quy định trực tiếp về vấn đề của mình (ví dụ: cùng là thủ tục khai sinh nhưng mỗi trường hợp có thể có quy định cụ thể khác nhau).

Bài toán đặt ra là xây dựng một trợ lý ảo thông minh có khả năng giúp người dùng xác định chính xác văn bản pháp lý phù hợp nhất và giải đáp chi tiết dựa trên nội dung văn bản đó thay vì đưa ra các câu trả lời chung chung, thiếu căn cứ.

1.2. Các hướng tiếp cận và hạn chế

Hiện nay có hai hướng tiếp cận chính:

1. Tìm kiếm từ khóa (Keyword Search - Full-text search):

- Ưu điểm: Chính xác tuyệt đối về mặt ký tự, tốc độ nhanh.
- Hạn chế: Trả về quá nhiều kết quả (hàng chục văn bản), buộc người dùng phải tự đọc rà soát từng file để tìm thông tin.

2. Mô hình ngôn ngữ lớn (Large Language Model – LLM), tiêu biểu như ChatGPT hoặc Gemini:

- Ưu điểm: Hiểu ngôn ngữ tự nhiên, trả lời mượt mà.
- Hạn chế: Tri thức bị "đóng băng" (bloated knowledge), không cập nhật văn bản mới và nghiêm trọng nhất là xu hướng "bịa đặt" (hallucination) khi trả lời các vấn đề chuyên môn sâu như luật.

1.3. Yêu cầu và tiêu chí của hệ thống đề xuất

Để giải quyết các thách thức trên, hệ thống đề xuất hướng tới các tiêu chí:

- Tính chính xác và có căn cứ: Mọi câu trả lời sinh ra phải dựa trên các đoạn văn bản pháp luật thực tế.

- Chiến lược tập trung nguồn tin (Single-Document Grounding): Thay vì cố gắng tổng hợp thông tin từ nhiều nguồn (dễ gây nhiễu và xung đột), hệ thống ưu tiên

chiến lược "chia để trị": xác định chính xác một văn bản nguồn (Single Source of Truth) phù hợp nhất với câu hỏi để trả lời sâu.

- Cơ chế gỡ rối chủ động: Hệ thống phải có khả năng phát hiện khi câu hỏi của người dùng mơ hồ (có thể ứng với nhiều văn bản khác nhau) và chủ động đưa ra các lựa chọn để người dùng xác nhận văn bản cần tra cứu.

- Khả năng trích dẫn (Citation): Chỉ rõ nguồn gốc thông tin được lấy từ văn bản pháp luật nào.

- Hiểu ngữ nghĩa (Semantic Understanding): Tìm kiếm dựa trên ý định thay vì chỉ khớp từ khóa.

- Quản lý phiên hội thoại (Session Management): Hệ thống cần duy trì trạng thái hội thoại để lưu trữ ngữ cảnh, văn bản nguồn đã được xác định và các lựa chọn của người dùng trong quá trình gỡ rối, đảm bảo tính nhất quán của câu trả lời xuyên suốt phiên làm việc.

1.4. Phạm vi triển khai và giới hạn

Trong khuôn khổ đề án này, hệ thống được triển khai tập trung vào các tính năng cốt lõi:

- Hệ thống Quản trị (Admin) để quản lý và cập nhật tri thức.
- Giao diện chatbot cho người dùng cuối để thực hiện hỏi đáp.

Giới hạn dữ liệu thử nghiệm ở các nhóm thủ tục hành chính phổ biến để đảm bảo tính khả thi trong đánh giá kết quả.

1.5. Kết luận chương

Chương 1 đã khái quát hóa nhu cầu cấp thiết của việc áp dụng trí tuệ nhân tạo (Artificial Intelligence – AI) vào lĩnh vực tra cứu pháp luật, phân tích những điểm yếu của các giải pháp cũ và đề ra hướng đi mới là kết hợp LLM với dữ liệu riêng (Retrieval-Augmented Generation – RAG), với chiến lược tập trung vào độ chính xác thông qua việc xác định văn bản nguồn duy nhất (Single-Document Grounding). Đây là tiền đề để đi vào nghiên cứu các cơ sở lý thuyết chi tiết ở Chương 2.

CHƯƠNG 2 NGHIÊN CỨU LÝ THUYẾT

2.1. Tổng quan RAG

Retrieval-Augmented Generation (RAG) là kiến trúc kết hợp giữa truy hồi thông tin (information retrieval) và mô hình sinh ngôn ngữ (language generation) nhằm tăng độ chính xác và tính kiểm chứng của hệ thống hỏi đáp dựa trên tri thức. Thay vì chỉ dựa vào kiến thức “nằm trong tham số” của mô hình, RAG cho phép truy xuất các đoạn văn bản liên quan từ một tập tài liệu (corpus) bên ngoài và dùng chúng làm ngữ cảnh (context) để sinh câu trả lời. Nhờ cơ chế “grounding” này, câu trả lời có thể bám sát nội dung nguồn, từ đó hạn chế hiện tượng bịa đặt thông tin (hallucination) và hỗ trợ trích dẫn căn cứ rõ ràng [1].

Động lực của RAG đến từ các hạn chế phổ biến của LLM khi sử dụng độc lập:

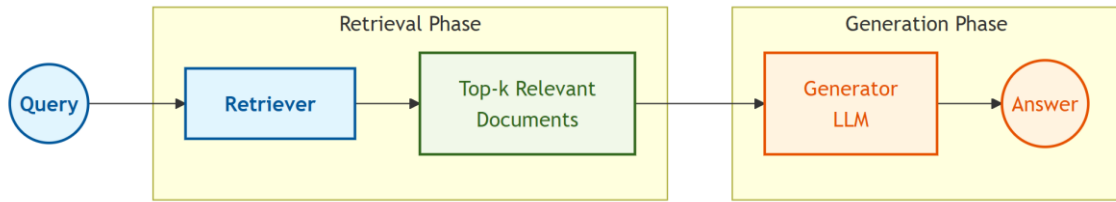
1. Mô hình có thể sinh ra nội dung nghe hợp lý nhưng không đúng với văn bản pháp lý thực tế.
2. Tri thức của mô hình bị giới hạn theo thời điểm huấn luyện, khó theo kịp thay đổi liên tục của văn bản pháp luật.

Trong bối cảnh hỏi đáp pháp luật, các hạn chế này trở nên nghiêm trọng vì yêu cầu bắt buộc về tính chính xác và căn cứ văn bản [1].

Về mặt kiến trúc, RAG được cấu thành bởi hai thành phần chính: Retriever và Generator. Retriever chịu trách nhiệm tìm các đoạn văn bản liên quan nhất từ kho tri thức dựa trên truy vấn, thường thông qua truy hồi ngữ nghĩa sử dụng vector embedding. Generator (thường là mô hình seq2seq/LLM) tiếp nhận câu hỏi kèm các đoạn truy hồi và sinh câu trả lời dựa trên ngữ cảnh này. Lewis et al. đề xuất hai biến thể quan trọng:

- RAG-Sequence: một tập đoạn truy hồi được dùng xuyên suốt quá trình sinh.
- RAG-Token: có thể dùng các đoạn truy hồi khác nhau theo từng token khi sinh, giúp tăng linh hoạt khi tổng hợp tri thức [1].

Quy trình hoạt động của RAG có thể mô tả như sau:



Hình 2.1. Quy trình hoạt động của mô hình RAG

Quy trình hoạt động của RAG được minh họa trong Hình 2.1, thể hiện sự kết hợp giữa pha truy hồi tri thức và pha sinh câu trả lời nhằm tận dụng đồng thời khả năng tìm kiếm ngữ nghĩa và khả năng sinh văn bản của mô hình ngôn ngữ.

Trước khi RAG được đề xuất, đã có nhiều nghiên cứu nhằm kết hợp truy hồi tri thức với mô hình ngôn ngữ. REALM, được Guu et al. đề xuất, tích hợp cơ chế truy hồi vào quá trình tiền huấn luyện để mô hình học cách sử dụng tri thức từ tập dữ liệu bên ngoài, tuy nhiên yêu cầu chi phí huấn luyện lớn và kém linh hoạt trong việc cập nhật tri thức [2]. Fusion-in-Decoder (FiD) của Izacard và Grave cải tiến khả năng xử lý nhiều đoạn văn bản truy hồi bằng cách kết hợp chúng tại decoder, song vẫn phụ thuộc vào tập đoạn truy hồi đầu vào và không tích hợp cơ chế truy hồi động trong quá trình sinh câu trả lời [3].

So với các phương pháp tiền nhiệm, RAG cung cấp một khuôn khổ linh hoạt nhờ việc tách biệt rõ ràng giữa kho tri thức và mô hình sinh, cho phép cập nhật tri thức thông qua dữ liệu mà không cần huấn luyện lại mô hình, đồng thời hỗ trợ trích dẫn nguồn văn bản cụ thể để tăng độ tin cậy. Thực nghiệm trên các bộ đánh giá chuẩn như Natural Questions, WebQuestions và CuratedTREC cho thấy RAG đạt hiệu quả vượt trội trong các bài toán hỏi đáp dựa trên tri thức [1].

2.2. Embedding và truy hồi ngữ nghĩa

2.2.1. Từ Word Embeddings đến Sentence Embeddings

Word embeddings là nền tảng của các kỹ thuật xử lý ngôn ngữ tự nhiên hiện đại, trong đó mỗi từ được biểu diễn bởi một vector số thực trong không gian nhiều chiều. Word2Vec, được giới thiệu bởi Mikolov et al., là một trong những phương pháp tiên phong với hai kiến trúc chính là Continuous Bag-of-Words (CBOW) và Skip-gram [4]. Mô hình này học các biểu diễn từ dựa trên giả thuyết phân bố (distributional hypothesis), theo đó các từ xuất hiện trong những ngữ cảnh tương tự

thường mang ý nghĩa tương tự.

Tuy nhiên, Word2Vec tồn tại một hạn chế quan trọng khi mỗi từ chỉ được gán một vector cố định, không phụ thuộc vào ngữ cảnh sử dụng. Điều này khiến mô hình không thể phân biệt các nghĩa khác nhau của cùng một từ trong các ngữ cảnh khác nhau. BERT (Bidirectional Encoder Representations from Transformers) của Devlin et al. đã khắc phục hạn chế này bằng cách tạo ra các biểu diễn từ theo ngữ cảnh (contextualized word embeddings) dựa trên kiến trúc Transformer [5]. Thông qua cơ chế self-attention hai chiều, mỗi từ được mã hóa dựa trên toàn bộ ngữ cảnh xung quanh, tạo ra các biểu diễn phong phú và linh hoạt hơn.

BERT được tiền huấn luyện với hai mục tiêu chính, bao gồm Masked Language Modeling (MLM) – dự đoán các từ bị che trong câu và Next Sentence Prediction (NSP) – xác định mối quan hệ liên tiếp giữa hai câu trong văn bản [5]. Mặc dù mang lại các biểu diễn từ theo ngữ cảnh hiệu quả, BERT ban đầu không được thiết kế tối ưu cho việc so sánh độ tương đồng ở mức câu, từ đó thúc đẩy sự ra đời của các phương pháp sentence embeddings trong các nghiên cứu tiếp theo.

2.2.2. Sentence-BERT cho Semantic Search

Mặc dù BERT tạo ra các biểu diễn ngữ nghĩa chất lượng cao, việc sử dụng trực tiếp BERT cho các bài toán đo độ tương đồng ngữ nghĩa gặp phải hai hạn chế lớn [6]. Thứ nhất, chi phí tính toán rất cao do việc so sánh hai câu yêu cầu đưa đồng thời cả hai câu qua mô hình BERT, khiến quá trình tìm kiếm trong tập dữ liệu lớn trở nên không khả thi. Ví dụ, với tập dữ liệu gồm 10.000 câu, phương pháp này cần thực hiện khoảng 50 triệu cặp suy luận. Thứ hai, mục tiêu huấn luyện Next Sentence Prediction (NSP) của BERT không được thiết kế để đo mức độ tương đồng ngữ nghĩa, mà chỉ đưa ra kết quả nhị phân về mối quan hệ giữa hai câu.

Sentence-BERT (SBERT), được đề xuất bởi Reimers và Gurevych, khắc phục các hạn chế trên bằng cách tinh chỉnh BERT theo kiến trúc siamese và triplet network [6]. Cách tiếp cận này cho phép mô hình sinh ra các vector biểu diễn câu có kích thước cố định, từ đó có thể so sánh trực tiếp bằng các độ đo như cosine similarity hoặc dot product.

Trong quá trình huấn luyện, SBERT sử dụng các hàm mục tiêu phù hợp cho bài toán đo độ tương đồng, bao gồm phân loại, hồi quy dựa trên cosine similarity và

triplet loss.

Nhờ đó, mỗi câu trong tập dữ liệu chỉ cần được mã hóa một lần thành embedding, sau đó quá trình tìm kiếm chỉ còn là phép so sánh vector, cho phép triển khai semantic search một cách hiệu quả trên các tập dữ liệu lớn [6].

2.2.3. Embedding Models cho Tiếng Việt

Việc áp dụng các mô hình embedding phổ biến như BERT và SBERT cho tiếng Việt gặp nhiều thách thức do đặc điểm ngôn ngữ đặc thù. Tiếng Việt là ngôn ngữ có hệ thống thanh điệu phong phú, trong đó dấu thanh đóng vai trò quan trọng trong việc phân biệt nghĩa của từ. Bên cạnh đó, nhiều khái niệm được biểu diễn dưới dạng từ ghép, chẳng hạn như “máy tính” hay “công an”, đòi hỏi quá trình tách từ chính xác. Ngoài ra, hiện tượng đa nghĩa phụ thuộc ngữ cảnh cũng làm tăng độ phức tạp trong việc học biểu diễn ngữ nghĩa [7].

PhoBERT, được đề xuất bởi Nguyen và Nguyen, là mô hình BERT được tiền huấn luyện riêng cho tiếng Việt trên tập dữ liệu lớn khoảng 20GB văn bản, bao gồm Wikipedia và các nguồn tin tức tiếng Việt [7]. Mô hình này sử dụng RDRSegmenter cho bước tách từ, giúp xử lý chính xác các từ ghép đặc trưng của tiếng Việt, kết hợp với phương pháp mã hóa Byte-Pair Encoding (BPE) với bộ từ vựng gồm 64.000 token. Về kiến trúc, PhoBERT giữ nguyên cấu hình tương tự BERT-base với 12 lớp Transformer, kích thước ẩn 768 và 12 đầu attention.

Thực nghiệm cho thấy PhoBERT đạt hiệu năng cao trên nhiều bài toán xử lý ngôn ngữ tiếng Việt như gán nhãn từ loại (POS tagging), nhận dạng thực thể (NER) và phân tích phụ thuộc cú pháp, qua đó khẳng định tầm quan trọng của việc tiền huấn luyện mô hình ngôn ngữ phù hợp cho từng ngôn ngữ cụ thể [7].

Bên cạnh các mô hình biểu diễn ở mức câu, Vietnamese-document-embedding của dangvantuan được thiết kế nhằm sinh embedding cho các văn bản tiếng Việt có độ dài lớn, với cửa sổ ngữ cảnh lên đến khoảng 8196 token. Mô hình được xây dựng trên kiến trúc Sentence-Transformers, kế thừa từ nền tảng gte-multilingual và được tối ưu cho bài toán truy hồi văn bản tiếng Việt thông qua quá trình huấn luyện và tinh chỉnh trên các bộ dữ liệu suy luận và đo độ tương đồng ngữ nghĩa tiếng Việt [8]. Nhờ khả năng sinh embedding có kích thước cố định cho các đoạn văn bản dài và hỗ trợ độ dài ngữ cảnh lớn, mô hình phù hợp để áp dụng trong các hệ thống truy hồi ngữ

nghĩa và Retrieval-Augmented Generation xử lý tài liệu tiếng Việt quy mô lớn.

Trong phạm vi đề tài, PhoBERT được trình bày như một mô hình nền tảng đại diện cho hướng tiền huấn luyện ngôn ngữ tiếng Việt. Hệ thống không sử dụng trực tiếp PhoBERT trong pipeline Retrieval-Augmented Generation.

2.2.4. Similarity Metrics và Retrieval Process

Sau khi các câu hoặc tài liệu được mã hóa thành vector embedding, bước tiếp theo trong truy hồi ngữ nghĩa là đo lường độ tương đồng giữa câu truy vấn và các ứng viên trong tập dữ liệu. Hai độ đo phổ biến nhất được sử dụng trong các mô hình sentence embedding là cosine similarity và dot product [6].

Cosine similarity đo góc giữa hai vector trong không gian nhiều chiều, được định nghĩa như sau:

$$\cos(u, v) = \frac{u \cdot v}{\|u\| \|v\|}$$

Độ đo này không bị ảnh hưởng bởi độ lớn của vector, phù hợp trong các trường hợp chỉ quan tâm đến hướng của biểu diễn ngữ nghĩa.

Dot product được định nghĩa là tích vô hướng giữa hai vector:

$$u \cdot v = \sum_{i=1}^n u_i v_i$$

So với cosine similarity, dot product có chi phí tính toán thấp hơn do không cần chuẩn hóa vector và có thể kết hợp cả thông tin về hướng và độ lớn của biểu diễn.

Quy trình truy hồi ngữ nghĩa trong các hệ thống semantic search thường bao gồm các bước: mã hóa toàn bộ tập tài liệu thành vector và lưu trữ trong cơ sở dữ liệu vector; mã hóa câu truy vấn đầu vào; tìm kiếm top-K tài liệu có độ tương đồng cao nhất; và tái xếp hạng kết quả bằng các mô hình phức tạp hơn nếu cần thiết. Nhờ sử dụng sentence embeddings, semantic search có khả năng truy xuất các tài liệu mang ý nghĩa tương tự ngay cả khi không có sự trùng khớp từ khóa chính xác, qua đó vượt trội so với các phương pháp tìm kiếm truyền thống như TF-IDF hay BM25 [6].

2.3. Vector database với PostgreSQL + pgvector

2.3.1. Vector database và Approximate Nearest Neighbor Search

Vector database là hệ thống lưu trữ và truy vấn được tối ưu hóa cho các vector có số chiều lớn, hỗ trợ hiệu quả các thao tác tìm kiếm láng giềng gần nhất (nearest neighbor search – NNS). Với tập tài liệu lớn chứa hàng triệu vector, việc tính toán độ tương đồng giữa vector truy vấn và toàn bộ vector trong cơ sở dữ liệu theo phương pháp brute-force trở nên không khả thi do độ phức tạp $O(n \cdot d)$ với n là số vectors và d là dimensionality.

Approximate Nearest Neighbor (ANN) search giải quyết vấn đề này bằng cách đánh đổi một phần độ chính xác để đạt được tốc độ truy vấn cao hơn, cho phép tìm kiếm xấp xỉ các láng giềng gần nhất với độ phức tạp dưới tuyến tính [9]. Các thuật toán ANN phổ biến bao gồm:

- Locality-Sensitive Hashing (LSH): ánh xạ các vector tương tự vào cùng một nhóm băm.
- Tree-based methods: KD-tree, Ball tree.
- Graph-based methods: HNSW (Hierarchical Navigable Small World)
- Clustering-based methods: IVF (Inverted File Index)

2.3.2. FAISS – Facebook AI Similarity Search

FAISS (Facebook AI Similarity Search) là một thư viện mã nguồn mở do Johnson et al. phát triển, nhằm phục vụ các bài toán tìm kiếm tương đồng và phân cụm trên các vector có số chiều lớn [10]. Thư viện này được thiết kế với mục tiêu tối ưu hiệu năng, cho phép thực hiện truy vấn gần đúng trên các tập dữ liệu có quy mô rất lớn.

FAISS hỗ trợ nhiều chiến lược lập chỉ mục khác nhau, cho phép đánh đổi linh hoạt giữa độ chính xác, tốc độ truy vấn và chi phí bộ nhớ, bao gồm các phương pháp tìm kiếm chính xác (brute-force) và các phương pháp tìm kiếm xấp xỉ dựa trên phân cụm hoặc đồ thị. Nhờ tối ưu bằng các tập lệnh SIMD và hỗ trợ tăng tốc bằng GPU, FAISS có thể xử lý hiệu quả các tập dữ liệu chứa hàng triệu đến hàng tỷ vector [10].

Tuy nhiên, FAISS hoạt động như một thư viện tìm kiếm vector độc lập và

không được tích hợp trực tiếp với các hệ quản trị cơ sở dữ liệu, do đó không hỗ trợ sẵn các đặc tính như giao dịch ACID, truy vấn SQL hay kiểm soát truy cập đồng thời. Điều này đặt ra thách thức khi triển khai FAISS trong các hệ thống sản xuất yêu cầu quản lý dữ liệu chặt chẽ.

2.3.3. pgvector - Vector Extension cho PostgreSQL

pgvector là một extension dành cho PostgreSQL, cung cấp kiểu dữ liệu vector và các toán tử cần thiết để thực hiện tìm kiếm tương đồng trong không gian vector [11]. Khác với các thư viện tìm kiếm vector độc lập như FAISS, pgvector được tích hợp trực tiếp vào hệ quản trị cơ sở dữ liệu PostgreSQL, cho phép thực hiện truy vấn vector thông qua ngôn ngữ SQL chuẩn.

Một trong những ưu điểm nổi bật của pgvector là khả năng tích hợp liền mạch với hệ sinh thái SQL. Người dùng có thể lưu trữ embedding dưới dạng một cột vector trong bảng quan hệ và thực hiện truy vấn tìm kiếm tương đồng trực tiếp bằng SQL. pgvector hỗ trợ ba toán tử chính cho đo độ tương đồng, bao gồm khoảng cách Euclid (L2 distance), inner product và cosine distance, phù hợp với các loại embedding phổ biến trong các hệ thống semantic search.

Bên cạnh đó, pgvector kế thừa đầy đủ các đặc tính ACID của PostgreSQL, bao gồm tính nguyên tử, nhất quán, cô lập và bền vững. Điều này đảm bảo tính toàn vẹn dữ liệu trong các môi trường sản xuất, đặc biệt trong các hệ thống nơi tài liệu có thể được thêm, sửa hoặc xóa trong khi vẫn phục vụ các truy vấn tìm kiếm ngữ nghĩa.

Ngoài khả năng tìm kiếm vector, pgvector còn cho phép kết hợp truy vấn ngữ nghĩa với các điều kiện quan hệ truyền thống thông qua các phép nối (JOIN) và mệnh đề lọc trong SQL. Nhờ đó, hệ thống có thể thực hiện các truy vấn semantic search có điều kiện, chẳng hạn như giới hạn theo lĩnh vực, thời gian hoặc các thuộc tính metadata khác. Khả năng này khiến pgvector trở thành một lựa chọn phù hợp cho các hệ thống Retrieval-Augmented Generation trong môi trường sản xuất, nơi yêu cầu đồng thời cả hiệu năng truy hồi và khả năng quản lý dữ liệu chặt chẽ [11].

2.3.4. Indexing Strategies trong pgvector

pgvector hỗ trợ các chiến lược lập chỉ mục nhằm tăng hiệu năng truy vấn tương đồng trong không gian vector, tiêu biểu là IVFFlat và HNSW [11]. IVFFlat là phương

pháp dựa trên phân cụm, trong đó các vector được chia thành nhiều cụm và quá trình truy vấn chỉ được thực hiện trên một tập con các cụm gần nhất. Cách tiếp cận này giúp giảm chi phí tìm kiếm so với việc quét toàn bộ không gian vector.

HNSW là phương pháp lập chỉ mục dựa trên đồ thị với cấu trúc nhiều tầng, cho phép tìm kiếm láng giềng gần nhất với độ trễ thấp và hiệu năng cao. Phương pháp này đạt hiệu quả tốt trong các kịch bản yêu cầu thông lượng truy vấn lớn, đổi lại là chi phí bộ nhớ và thời gian xây dựng chỉ mục cao hơn so với các phương pháp dựa trên phân cụm [11].

Việc lựa chọn chiến lược lập chỉ mục phù hợp phụ thuộc vào quy mô dữ liệu, yêu cầu độ chính xác và đặc tính tải của hệ thống. Các tham số cấu hình cụ thể và chiến lược tối ưu hóa truy vấn với pgvector sẽ được trình bày chi tiết trong chương hiện thực hóa nghiên cứu.

2.3.5. So sánh FAISS vs pgvector

Tiêu chí	FAISS	pgvector
Tốc độ search	Rất nhanh, GPU support	Nhanh, CPU-only
Scale	Hàng tỷ vector	Millions of vectors
ACID	Không hỗ trợ	Đầy đủ ACID
SQL integration	Không	Native PostgreSQL
Memory usage	Cao (PQ có thể compress)	Trung bình
Concurrent writes	Phức tạp	Native support
Ease of use	Cần setup riêng	Chỉ cần PostgreSQL extension
Production-ready	Cần wrapper layer	Sẵn sàng

Bảng 2.1. So sánh FAISS vs pgvector

Từ bảng so sánh FAISS và pgvector (Bảng 2.1), có thể thấy mỗi giải pháp phù hợp với các bối cảnh sử dụng khác nhau. Đối với hệ thống Retrieval-Augmented Generation trong lĩnh vực pháp luật, nơi tập dữ liệu có quy mô vừa, yêu cầu đảm bảo tính nhất quán dữ liệu, hỗ trợ truy vấn theo metadata và triển khai đơn giản, pgvector là lựa chọn phù hợp hơn. Việc tích hợp trực tiếp với PostgreSQL cho phép hệ thống tận dụng các đặc tính ACID và khả năng truy vấn SQL, đáp ứng tốt yêu cầu của một hệ thống tìm kiếm ngữ nghĩa trong môi trường sản xuất [10], [11].

2.4. Reranking với Cross-Encoder

2.4.1. Bối cảnh two-stage retrieval

Trong các hệ thống truy hồi ngữ nghĩa hiện đại, đặc biệt là Retrieval-Augmented Generation, chiến lược two-stage retrieval được sử dụng phổ biến nhằm cân bằng giữa hiệu năng và độ chính xác. Quy trình này bao gồm hai giai đoạn chính: (i) sử dụng bi-encoder (dense retriever) để truy hồi nhanh một tập ứng viên từ tập dữ liệu lớn và (ii) áp dụng cross-encoder để tái xếp hạng (reranking) các ứng viên này nhằm nâng cao độ chính xác của kết quả cuối cùng [12].

Trong kiến trúc này, bi-encoder được tối ưu cho tốc độ truy vấn thông qua việc mã hóa độc lập câu truy vấn và tài liệu, trong khi cross-encoder tập trung tối đa vào độ chính xác. Mục tiêu của giai đoạn reranking là chuyển đổi tập kết quả có độ bao phủ cao từ retriever thành một tập nhỏ hơn với độ chính xác cao hơn ở các vị trí đầu.

2.4.2. Cơ chế hoạt động của Cross-Encoder

Khác với bi-encoder, cross-encoder xử lý câu truy vấn và tài liệu trong cùng một chuỗi đầu vào. Cụ thể, truy vấn và văn bản tài liệu được ghép lại thành một chuỗi duy nhất và đưa qua mô hình Transformer, cho phép cơ chế self-attention khai thác đầy đủ các tương tác ở mức token giữa hai thành phần này. Đầu ra của mô hình là một điểm số thể hiện mức độ liên quan giữa truy vấn và tài liệu.

Điểm số này thường được suy ra từ embedding đại diện của toàn bộ chuỗi đầu vào (như token [CLS] hoặc trung bình các embedding) thông qua một tầng tuyến tính đơn giản [12]. Nhờ khả năng mô hình hóa đầy đủ ngữ cảnh chung, cross-encoder có thể đưa ra đánh giá chính xác hơn so với các phương pháp mã hóa độc lập.

Tuy nhiên, do mỗi cặp truy vấn - tài liệu đều phải được suy luận riêng biệt, cross-encoder có chi phí tính toán cao và không phù hợp để áp dụng trực tiếp trên toàn bộ tập dữ liệu.

2.4.3. So sánh Bi-Encoder và Cross-Encoder

Bi-encoder và cross-encoder có những đặc điểm bổ trợ cho nhau. Bi-encoder mã hóa truy vấn và tài liệu một cách độc lập, cho phép sử dụng các kỹ thuật ANN để tìm kiếm nhanh trên tập dữ liệu lớn, nhưng đánh đổi bằng việc bỏ qua các tương tác chi tiết ở mức token. Ngược lại, cross-encoder mô hình hóa đầy đủ tương tác token-

level giữa truy vấn và tài liệu, mang lại độ chính xác cao hơn, nhưng có chi phí suy luận lớn [12].

Do đó, cross-encoder thường được sử dụng để tái xếp hạng một tập ứng viên đã được truy hồi trước đó, nhằm chọn ra các tài liệu có mức độ liên quan cao nhất để cung cấp cho mô hình sinh câu trả lời.

2.4.4. BGE Reranker cho bài toán đa ngữ

Trong số các mô hình cross-encoder được sử dụng cho bài toán reranking, BGE reranker là một lựa chọn phổ biến trong các hệ thống đa ngữ. Mô hình này được thiết kế và huấn luyện trực tiếp cho nhiệm vụ xếp hạng mức độ liên quan giữa truy vấn và tài liệu, đồng thời hỗ trợ nhiều ngôn ngữ khác nhau, bao gồm cả tiếng Việt.

BGE reranker thường trả về điểm số đã được chuẩn hóa, thuận tiện cho việc so sánh và kết hợp với các tín hiệu khác trong hệ thống. So với các mô hình BERT kích thước lớn, BGE reranker có kiến trúc gọn hơn nhưng vẫn đạt hiệu quả cao, đồng thời dễ tích hợp vào pipeline của các hệ thống Retrieval-Augmented Generation [12].

2.5. Context Expansion và Citation Tracing

Trong hệ thống Retrieval-Augmented Generation, việc quản lý và cung cấp ngữ cảnh phù hợp đóng vai trò then chốt trong việc đảm bảo chất lượng câu trả lời. Khi tài liệu được chia thành các đoạn nhỏ nhằm tối ưu cho truy hồi, thông tin quan trọng có thể bị tách rời hoặc phân tán ở nhiều đoạn khác nhau, đặc biệt trong các văn bản pháp luật có cấu trúc tham chiếu chéo phức tạp [1]. Do đó, việc quản lý ngữ cảnh (context) hiệu quả là yêu cầu cần thiết để mô hình tạo sinh có thể tổng hợp và diễn giải thông tin chính xác.

Thay vì chia tài liệu thuần túy theo số lượng token, section-based chunking tận dụng cấu trúc logic của văn bản để đảm bảo tính toàn vẹn ngữ nghĩa của từng phần [1]. Đối với văn bản pháp luật Việt Nam, cấu trúc phân cấp như chương, điều và khoản giúp việc chia đoạn theo section giữ nguyên nội dung pháp lý hoàn chỉnh, đồng thời hỗ trợ truy xuất và trích dẫn chính xác nguồn gốc thông tin. Cách tiếp cận này đặc biệt phù hợp với các hệ thống Legal RAG, nơi yêu cầu cao về độ chính xác và khả năng kiểm chứng.

Trong một số trường hợp, việc chỉ sử dụng các đoạn văn bản được truy hồi

ban đầu có thể chưa đủ để trả lời đầy đủ câu hỏi. Khi đó, các kỹ thuật mở rộng ngữ cảnh (context expansion) có thể được áp dụng, bao gồm chồng lấn đoạn (chunk overlap), truy hồi các đoạn lân cận hoặc truy hồi theo cấu trúc phân cấp. Các kỹ thuật này giúp cung cấp ngữ cảnh rộng hơn cho mô hình sinh, tuy nhiên cần được cân nhắc do có thể làm tăng chi phí tính toán và độ phức tạp của hệ thống [1].

Bên cạnh việc mở rộng ngữ cảnh, citation tracing đề cập đến khả năng truy vết và liên kết câu trả lời sinh ra với các đoạn văn bản nguồn cụ thể đã được truy hồi. Cơ chế này giúp tăng tính minh bạch và khả năng kiểm chứng của hệ thống, đặc biệt quan trọng trong các ứng dụng pháp luật, nơi câu trả lời cần đi kèm căn cứ rõ ràng từ văn bản quy phạm pháp luật [1].

2.6. LLM và chiến lược sinh câu trả lời

2.6.1. Tổng quan về Large Language Models

Large Language Models (LLMs) là các mô hình ngôn ngữ quy mô lớn được huấn luyện trước trên tập dữ liệu văn bản rất lớn, có khả năng sinh văn bản tự nhiên và thực hiện nhiều tác vụ xử lý ngôn ngữ khác nhau. Phần lớn các LLM hiện đại được xây dựng theo kiến trúc Transformer và huấn luyện với mục tiêu dự đoán token tiếp theo, cho phép mô hình học được các mẫu ngữ pháp, ngữ nghĩa và kiến thức ngôn ngữ tổng quát từ dữ liệu [13].

Một đặc điểm quan trọng của LLM là khả năng in-context learning, trong đó mô hình có thể thực hiện một nhiệm vụ mới chỉ dựa trên hướng dẫn và ví dụ được cung cấp trong prompt, mà không cần huấn luyện lại. Tuy nhiên, do kiến thức của LLM bị giới hạn trong dữ liệu huấn luyện và không có cơ chế kiểm chứng thông tin bên ngoài, các mô hình này dễ gặp hiện tượng sinh thông tin không chính xác khi được sử dụng độc lập.

Trong kiến trúc Retrieval-Augmented Generation, LLM đóng vai trò là bộ sinh câu trả lời (generator), nhận đầu vào là câu hỏi cùng với ngữ cảnh được truy hồi từ hệ thống tìm kiếm. Nhờ việc kết hợp ngữ cảnh bên ngoài, LLM có thể tổng hợp thông tin một cách có căn cứ, giảm hiện tượng hallucination và nâng cao độ chính xác của câu trả lời.

Hiện nay, các LLM được sử dụng trong các hệ thống RAG có thể được phân

thành hai nhóm chính: các mô hình thương mại quy mô lớn được cung cấp thông qua API (ví dụ các mô hình của OpenAI hoặc Google) và các mô hình mã nguồn mở có thể triển khai cục bộ. Các mô hình thương mại thường có năng lực suy luận mạnh và hỗ trợ cửa sổ ngữ cảnh lớn, trong khi các mô hình mã nguồn mở mang lại lợi thế về khả năng kiểm soát dữ liệu và chi phí triển khai [14]. Việc lựa chọn LLM cụ thể phụ thuộc vào yêu cầu của hệ thống, bao gồm độ chính xác, độ trễ, chi phí và ràng buộc về dữ liệu.

2.6.2. In-Context Learning và Few-Shot Prompting

In-context learning là khả năng của các mô hình ngôn ngữ lớn học và thực hiện một tác vụ mới thông qua các ví dụ được cung cấp trực tiếp trong prompt, mà không cần cập nhật hay tinh chỉnh tham số mô hình [13]. Cơ chế này cho phép LLMs thích nghi linh hoạt với nhiều bài toán khác nhau thông qua cách thiết kế ngữ cảnh đầu vào.

Brown et al. phân biệt ba thiết lập phổ biến của in-context learning. Ở chế độ zero-shot, mô hình chỉ nhận mô tả nhiệm vụ mà không có ví dụ minh họa. Trong thiết lập one-shot, mô hình được cung cấp một ví dụ duy nhất để suy luận cách thực hiện tác vụ. Thiết lập few-shot mở rộng cách tiếp cận này bằng cách cung cấp nhiều ví dụ trong prompt, thường mang lại hiệu năng tốt hơn so với zero-shot và one-shot [13].

Thực nghiệm của Brown et al. cho thấy hiệu năng của mô hình tăng đáng kể khi số lượng ví dụ trong prompt tăng lên, đặc biệt trong khoảng từ vài ví dụ đến khoảng 10–20 ví dụ, sau đó có xu hướng bão hòa. Với GPT-3 quy mô 175 tỷ tham số, phương pháp few-shot prompting trong nhiều trường hợp đạt hiệu năng tiệm cận, thậm chí vượt qua các mô hình được fine-tune truyền thống trên nhiều tác vụ xử lý ngôn ngữ tự nhiên [13].

Trong bối cảnh hệ thống Retrieval-Augmented Generation, in-context learning và few-shot prompting đóng vai trò quan trọng trong việc hướng dẫn mô hình sinh câu trả lời dựa trên ngữ cảnh truy hồi, định dạng đầu ra mong muốn và cách trích dẫn thông tin, mà không cần huấn luyện lại mô hình sinh.

2.7. Session Management & Conversational RAG

2.7.1. Từ Single-turn đến Multi-turn Conversation

Các hệ thống RAG ban đầu được thiết kế cho bài toán hỏi đáp đơn lượt (single-turn), trong đó mỗi câu hỏi được xử lý độc lập và không phụ thuộc vào các lượt tương tác trước đó [1]. Tuy nhiên, trong thực tế, người dùng thường đặt chuỗi câu hỏi liên quan theo ngữ cảnh hội thoại, đòi hỏi hệ thống phải duy trì và khai thác thông tin từ các lượt trao đổi trước.

Conversational RAG mở rộng kiến trúc RAG truyền thống bằng cách duy trì trạng thái hội thoại (conversation state) xuyên suốt nhiều lượt hỏi đáp, cho phép hệ thống hiểu và xử lý hiệu quả các câu hỏi tiếp nối (follow-up questions) [1], [13].

2.7.2. Session State và Quản lý Ngữ cảnh Hội thoại

Trong Conversational RAG, session đóng vai trò là đơn vị quản lý một phiên tương tác giữa người dùng và hệ thống. Trạng thái session thường bao gồm các thông tin như câu hỏi gần nhất, tài liệu đang được quan tâm và lịch sử hội thoại rút gọn. Việc duy trì session giúp hệ thống hiểu được mối liên hệ giữa các câu hỏi liên tiếp, đồng thời tránh việc truy hồi lại toàn bộ tập tài liệu cho mỗi lượt tương tác.

Đối với các hệ thống RAG trong lĩnh vực pháp luật, việc lưu trữ trạng thái hội thoại có ý nghĩa quan trọng nhằm đảm bảo tính nhất quán, khả năng truy vết và phục vụ yêu cầu kiểm tra sau này.

2.7.3. Context Pinning, xử lý Follow-up Questions và Contextualization

Context pinning là một chiến lược phổ biến trong Conversational RAG, trong đó hệ thống “ghim” một tài liệu hoặc một phạm vi tài liệu cụ thể vào session khi người dùng thể hiện sự quan tâm đến một văn bản nhất định [1]. Các câu hỏi tiếp theo sẽ ưu tiên truy hồi và sinh câu trả lời dựa trên ngữ cảnh đã được ghim, thay vì tìm kiếm trên toàn bộ corpus.

Cách tiếp cận này đặc biệt phù hợp với các văn bản pháp luật, nơi người dùng thường đặt nhiều câu hỏi xoay quanh cùng một văn bản, điều khoản hoặc thông tư cụ thể. Context pinning giúp tăng độ liên quan của kết quả, cải thiện tốc độ truy hồi và hạn chế hiện tượng lệch chủ đề trong hội thoại.

Các câu hỏi tiếp nối trong hội thoại thường không đầy đủ ngữ cảnh nếu xét

độc lập. Do đó, Conversational RAG cần cơ chế nhận diện câu hỏi follow-up và bổ sung ngữ cảnh phù hợp trước khi thực hiện truy hồi và sinh câu trả lời [1]. Các hướng tiếp cận phổ biến bao gồm sử dụng trạng thái session, tái cấu trúc câu hỏi dựa trên ngữ cảnh trước đó hoặc giới hạn phạm vi tìm kiếm trong các tài liệu đã được ghim.

2.8. Tối ưu hóa cho tiếng Việt

2.8.1. Thách thức ngôn ngữ đối với truy hồi ngữ nghĩa

Việc xây dựng hệ thống RAG cho tiếng Việt gặp nhiều thách thức do đặc điểm ngôn ngữ. Thứ nhất, tiếng Việt sử dụng dấu thanh, trong đó dấu có vai trò phân biệt nghĩa và ảnh hưởng trực tiếp đến biểu diễn ngữ nghĩa. Thứ hai, khoảng trắng trong tiếng Việt chủ yếu phân tách âm tiết thay vì từ, khiến bài toán tách từ (word segmentation) trở thành bước tiền xử lý quan trọng. Ngoài ra, tiếng Việt có nhiều từ ghép và hiện tượng đa nghĩa phụ thuộc ngữ cảnh, đòi hỏi các mô hình biểu diễn theo ngữ cảnh (contextualized embeddings) để giảm nhiễu trong truy hồi [7].

2.8.2. Tokenization, word segmentation và lựa chọn mô hình

Các tokenizer phổ biến được thiết kế chủ yếu cho tiếng Anh có thể không tối ưu khi áp dụng trực tiếp cho tiếng Việt, do dễ tách sai đơn vị từ và làm suy giảm thông tin ngữ nghĩa. PhoBERT là một mô hình tiền huấn luyện theo hướng đơn ngữ cho tiếng Việt, được xây dựng trên văn bản tiếng Việt quy mô lớn và sử dụng quy trình tách từ trước khi huấn luyện, nhờ đó cải thiện chất lượng biểu diễn ngữ nghĩa so với các mô hình đa ngữ trong nhiều tác vụ tiếng Việt [7]. Trong bối cảnh RAG, việc sử dụng các mô hình embedding phù hợp ngôn ngữ giúp tăng chất lượng truy hồi, đặc biệt với các truy vấn có tính chuyên ngành hoặc cấu trúc pháp lý.

2.8.3. Lưu ý tiền xử lý cho văn bản pháp luật tiếng Việt

Văn bản pháp luật có cấu trúc phân cấp rõ ràng (chương, điều, khoản, điểm) và chứa nhiều tham chiếu chéo. Do đó, tiền xử lý và chunking cần ưu tiên bảo toàn cấu trúc pháp lý để tránh làm mất ngữ cảnh quan trọng. Ngoài ra, chuẩn hóa Unicode và giữ nguyên các mẫu tham chiếu (ví dụ “Điều”, “Khoản”, số hiệu văn bản) giúp giảm lỗi tokenization và cải thiện độ ổn định của truy hồi. Các lựa chọn tối ưu cụ thể (mô hình embedding, chiến lược chunking và chuẩn hóa) sẽ được trình bày trong chương triển khai hệ thống.

2.9. Kiến trúc Microservices

2.9.1. Từ Monolithic đến Microservices

Kiến trúc monolithic là cách tiếp cận truyền thống, trong đó toàn bộ hệ thống được triển khai như một khối thống nhất. Với hệ thống RAG, monolithic thường gom chung các thành phần như xử lý tài liệu, embedding, truy hồi vector và sinh câu trả lời vào cùng một ứng dụng [15]. Cách tiếp cận này đơn giản khi hệ thống nhỏ, nhưng gặp hạn chế khi mở rộng: khó scale theo từng thành phần, thay đổi một phần nhỏ có thể buộc phải triển khai lại toàn bộ và khó tối ưu tài nguyên do mỗi mô-đun có yêu cầu khác nhau (ví dụ embedding/LLM có thể cần GPU trong khi truy hồi chỉ cần CPU để tối ưu).

Microservices giải quyết bằng cách tách hệ thống thành nhiều dịch vụ nhỏ, mỗi dịch vụ đảm nhiệm một năng lực riêng và có thể triển khai độc lập [15]. Điều này phù hợp với RAG vì các thành phần trong pipeline có mức tải và yêu cầu tài nguyên khác nhau.

2.9.2. Các đặc trưng chính của Microservices và áp dụng vào hệ thống

Theo Newman, microservices thường có các đặc trưng như: (i) dịch vụ tập trung vào một trách nhiệm rõ ràng, (ii) triển khai độc lập và (iii) giao tiếp thông qua API thay vì gọi trực tiếp trong cùng tiến trình [15]. Ngoài ra, mỗi dịch vụ có thể lựa chọn công nghệ phù hợp với yêu cầu riêng, giúp tăng tính linh hoạt khi phát triển và tối ưu vận hành.

Trong phạm vi hệ thống RAG, việc phân rã theo năng lực chức năng giúp tách rõ các thành phần cốt lõi. Có thể phân rã hệ thống RAG thành các dịch vụ chức năng như: dịch vụ embedding thực hiện chuyển đổi văn bản thành vector; dịch vụ truy hồi thực hiện tìm kiếm tương đồng trên vector database; dịch vụ sinh câu trả lời đảm nhiệm việc tạo phản hồi dựa trên ngữ cảnh truy hồi; và dịch vụ lưu trữ quản lý tài liệu đầu vào [15]. Cách tổ chức này cho phép mở rộng độc lập theo nhu cầu (chẳng hạn scale embedding hoặc LLM khi tải tăng), đồng thời giảm rủi ro khi cập nhật từng thành phần.

Bên cạnh ưu điểm, microservices cũng làm tăng độ phức tạp vận hành do phát sinh giao tiếp qua mạng, giám sát và quản lý lỗi giữa các dịch vụ.

2.10. Object Storage (MinIO)

Trong hệ thống Retrieval-Augmented Generation, đặc biệt với dữ liệu pháp luật, việc lưu trữ tài liệu gốc (PDF, DOCX) cần đáp ứng các yêu cầu về khả năng mở rộng, truy cập linh hoạt và quản lý metadata hiệu quả. So với các hệ thống file truyền thống, object storage cung cấp mô hình lưu trữ phẳng, truy cập qua giao thức HTTP và hỗ trợ metadata phong phú, phù hợp hơn với các hệ thống cloud-native và các pipeline xử lý tài liệu quy mô lớn.

Object storage cho phép lưu trữ số lượng lớn tài liệu với khả năng mở rộng gần như không giới hạn, hỗ trợ truy cập trực tiếp thông qua các API tiêu chuẩn (REST/S3), gắn metadata cho từng đối tượng và quản lý phiên bản tài liệu theo thời gian. Những đặc tính này đặc biệt quan trọng trong các hệ thống Legal RAG, nơi cần truy vết nguồn gốc văn bản, hỗ trợ trích dẫn và quản lý các phiên bản văn bản pháp luật được sửa đổi hoặc thay thế.

MinIO là một hệ thống object storage tương thích S3, được thiết kế tối ưu cho các ứng dụng cloud-native và triển khai on-premise [AISTor – MinIO]. Các hệ thống object storage tương thích S3 như MinIO thường được sử dụng làm tầng lưu trữ tài liệu gốc nhờ khả năng tích hợp thuận tiện với các SDK, hỗ trợ metadata và versioning, cũng như khả năng triển khai linh hoạt trong kiến trúc microservices.

Trong kiến trúc tổng thể của hệ thống RAG, object storage đóng vai trò là tầng lưu trữ tài liệu gốc, tách biệt khỏi các thành phần xử lý embedding và truy hồi. Trong hệ thống, MinIO được sử dụng làm tầng object storage tương thích S3.

2.11. Phương pháp đánh giá

Đánh giá là bước cần thiết nhằm xác định mức độ hiệu quả của hệ thống Retrieval-Augmented Generation trong việc truy hồi các đoạn văn bản pháp luật liên quan phục vụ sinh câu trả lời. Trong phạm vi đề tài, nghiên cứu tập trung vào đánh giá chất lượng truy hồi (retrieval quality), vì đây là thành phần trực tiếp ảnh hưởng đến độ chính xác và tính kiểm chứng của câu trả lời sinh ra.

Trong các hệ thống RAG, kết quả truy hồi thường được sắp xếp theo mức độ liên quan và chỉ K đoạn văn bản đứng đầu (top-K) được sử dụng làm ngữ cảnh đầu vào cho mô hình ngôn ngữ lớn. Do đó, các chỉ số đánh giá trong nghiên cứu này được

xét trong phạm vi top-K kết quả truy hồi, ký hiệu là @K, nhằm phản ánh đúng chất lượng của tập ngữ cảnh thực sự ảnh hưởng đến quá trình sinh câu trả lời.

Việc đánh giá được thực hiện dựa trên các chỉ số chuẩn trong Information Retrieval, bao gồm Precision@K, Recall@K và F1@K [16]. Các chỉ số này được định nghĩa theo công thức như sau:

$$Precision = \frac{TP}{TP + FP}$$

$$Recall = \frac{TP}{TP + FN}$$

$$F1 - score = \frac{2 \cdot Precision \cdot Recall}{Precision + Recall}$$

Trong đó, TP là số lượng đoạn văn bản liên quan được truy hồi đúng, FP là số đoạn văn bản không liên quan nhưng được truy hồi và FN là số đoạn văn bản liên quan nhưng không được truy hồi. Trong nghiên cứu này, các giá trị TP, FP và FN được tính trong phạm vi top-K kết quả truy hồi cho mỗi truy vấn.

Cụ thể, Precision@K phản ánh mức độ chính xác của tập kết quả truy hồi được sử dụng làm ngữ cảnh đầu vào cho mô hình sinh; Recall@K phản ánh mức độ bao phủ các đoạn văn bản liên quan trong top-K so với tập ground truth; và F1@K được sử dụng như một chỉ số tổng hợp nhằm cân bằng giữa Precision@K và Recall@K khi so sánh hiệu quả của các cấu hình truy hồi khác nhau.

Ground truth cho quá trình đánh giá được xây dựng dưới dạng nhãn nhị phân (relevant / not relevant) cho mỗi truy vấn, dựa trên đánh giá của người có chuyên môn trong lĩnh vực ứng dụng. Kết quả đánh giá được tổng hợp trên toàn bộ tập truy vấn nhằm phản ánh hiệu năng chung của hệ thống. Việc áp dụng các chỉ số đánh giá này cho bài toán hỏi đáp pháp luật sẽ được trình bày chi tiết trong Chương 4.

2.12. Kết luận chương

Chương này đã trình bày các cơ sở lý thuyết và các nghiên cứu liên quan phục vụ cho việc xây dựng hệ thống Retrieval-Augmented Generation trong lĩnh vực hỏi đáp pháp luật. Nội dung chương tập trung làm rõ kiến trúc RAG, các phương pháp biểu diễn và truy hồi ngữ nghĩa, cơ sở dữ liệu vector, cơ chế tái xếp hạng, quản lý ngữ cảnh, cũng như các lựa chọn kiến trúc và phương pháp đánh giá phù hợp. Các

khái niệm và phương pháp được trình bày trong chương này đóng vai trò nền tảng cho quá trình thiết kế và hiện thực hóa hệ thống. Trên cơ sở đó, Chương 3 sẽ trình bày chi tiết quá trình phân tích yêu cầu, thiết kế kiến trúc và hiện thực hóa hệ thống RAG được đề xuất.

CHƯƠNG 3 HIỆN THỰC HÓA NGHIÊN CỨU

3.1. Phân tích yêu cầu

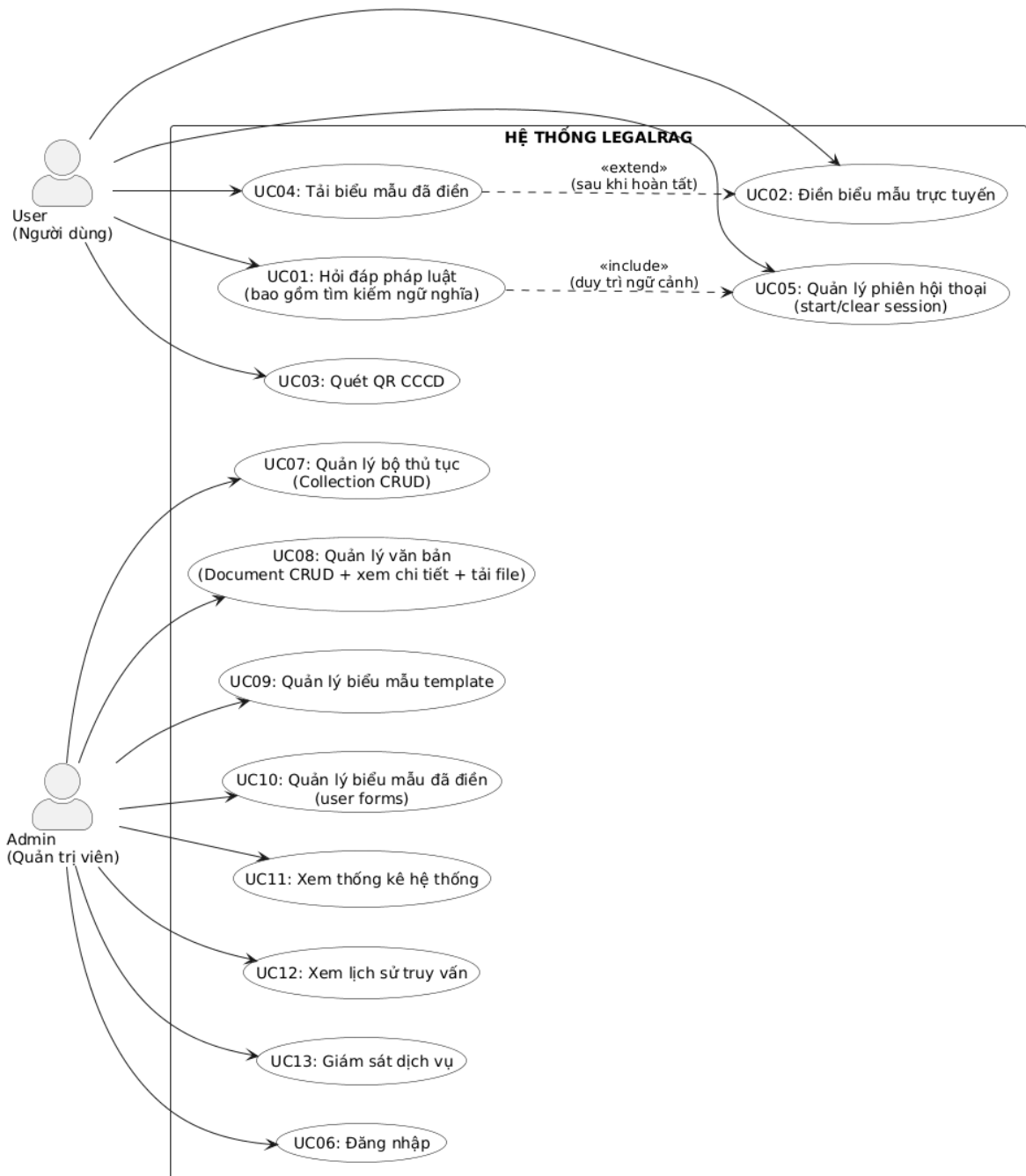
3.1.1. Actor và use-case

Hệ thống LegalRAG được xây dựng nhằm phục vụ hai nhóm đối tượng người dùng chính, bao gồm Người dùng (User) và Quản trị viên (Admin). Mỗi nhóm được xác định với vai trò và phạm vi chức năng riêng, tương ứng với các yêu cầu nghiệp vụ và mức độ truy cập khác nhau trong hệ thống.

Người dùng (User) là đối tượng sử dụng hệ thống để tra cứu thông tin pháp luật, đặt câu hỏi liên quan đến thủ tục hành chính và thực hiện điền các biểu mẫu pháp lý. Nhằm đảm bảo tính thuận tiện và phù hợp với đặc thù của hoạt động tra cứu pháp luật công khai, hệ thống cho phép người dùng truy cập và sử dụng các chức năng này mà không cần thực hiện đăng nhập hay xác thực danh tính.

Quản trị viên (Admin) là đối tượng chịu trách nhiệm quản lý và vận hành hệ thống, bao gồm quản lý kho văn bản pháp luật, quản lý biểu mẫu, theo dõi lịch sử truy vấn và giám sát trạng thái hoạt động của các dịch vụ. Do các chức năng này có ảnh hưởng trực tiếp đến dữ liệu và quá trình vận hành của hệ thống, quản trị viên bắt buộc phải đăng nhập và được xác thực thông qua cơ chế cấp JWT.

Trên cơ sở hai actor đã xác định, hệ thống được đặc tả bằng tập hợp các use-case tương ứng với các chức năng nghiệp vụ chính. Các use-case của người dùng tập trung vào khai thác và sử dụng tri thức pháp luật, trong khi các use-case của quản trị viên tập trung vào quản lý dữ liệu và giám sát hoạt động hệ thống. Sơ đồ use-case tổng quan minh họa mối quan hệ giữa các actor và các chức năng của hệ thống được trình bày trong Hình 3.1.



Hình 3.1. Sơ đồ use-case

Hình 3.1 mô tả các chức năng nghiệp vụ chính của hệ thống theo hai nhóm actor. Trong đó, các use-case phía người dùng tập trung vào hỏi đáp pháp luật, điền biểu mẫu và quản lý phiên hội thoại; các use-case phía quản trị viên tập trung vào quản lý dữ liệu và giám sát vận hành. Ký hiệu <<include>> thể hiện use-case bắt buộc được thực hiện như một phần của use-case khác (ví dụ: các chức năng quản lý phiên yêu cầu tồn tại ngữ cảnh hội thoại), trong khi <<extend>> thể hiện hành vi mở rộng chỉ xảy ra khi có điều kiện (ví dụ: Tải biểu mẫu đã điền sau khi đã nhập liệu).

3.1.2. Yêu cầu chức năng

Nhằm làm rõ các chức năng nghiệp vụ tương ứng với từng tác nhân, các use-case của hệ thống được phân loại và mô tả trong các bảng dưới đây:

Mã UC	Tên use-case	Mô tả	Đầu vào	Đầu ra
UC01	Hỏi đáp pháp luật	Người dùng đặt câu hỏi bằng ngôn ngữ tự nhiên; hệ thống tìm kiếm ngữ nghĩa trong kho văn bản pháp luật và sinh câu trả lời có trích dẫn nguồn.	Câu hỏi tiếng Việt	Câu trả lời + nguồn trích dẫn
UC02	Điền biểu mẫu trực tuyến	Người dùng điền thông tin vào biểu mẫu pháp lý thông qua giao diện web.	Thông tin biểu mẫu	Biểu mẫu ở dạng xem trước
UC03	Quét CCCD tự động điền	Hệ thống trích xuất thông tin cá nhân từ CCCD để hỗ trợ điền biểu mẫu.	Ảnh CCCD	Dữ liệu cá nhân
UC04	Tải biểu mẫu đã điền	Xuất biểu mẫu đã hoàn thiện để in ấn hoặc nộp hồ sơ.	Dữ liệu biểu mẫu	File biểu mẫu hoàn chỉnh
UC05	Quản lý phiên hội thoại	Khởi tạo hoặc xóa phiên hội thoại để thay đổi ngữ cảnh tra cứu.	Thông tin phiên	Phiên hội thoại mới

Bảng 3.1. Use-case cho Người dùng (User)

Mã UC	Tên use-case	Mô tả	Đầu vào	Đầu ra
UC06	Đăng nhập	Xác thực quản trị viên để truy cập các chức năng quản lý của hệ thống.	Thông tin đăng nhập	Trạng thái xác thực hợp lệ
UC07	Quản lý bộ thủ tục	Tạo mới, chỉnh sửa, xóa và quản lý các bộ thủ tục hành chính (collection) trong hệ thống.	Thông tin bộ thủ tục	Bộ thủ tục được cập nhật
UC08	Quản lý văn bản	Quản lý văn bản pháp luật bao gồm tải lên, xem chi tiết, chỉnh sửa thông tin và thay thế văn bản.	File văn bản và metadata	Văn bản được xử lý và lưu trữ
UC09	Quản lý biểu mẫu template	Quản lý các biểu mẫu pháp lý dạng template phục vụ việc điền biểu mẫu trực tuyến.	File biểu mẫu template	Biểu mẫu được cập nhật
UC10	Quản lý biểu mẫu đã điền	Theo dõi, tra cứu và quản lý các biểu mẫu đã được người dùng hoàn thiện.	Thông tin truy vấn	Danh sách biểu mẫu đã điền
UC11	Xem thống kê hệ thống	Xem các số liệu thống kê liên quan đến văn bản, truy vấn và mức độ sử dụng hệ thống.	Khoảng thời gian	Báo cáo thống kê
UC12	Xem lịch sử truy vấn	Tra cứu lịch sử các câu hỏi và phản hồi của người dùng trong hệ thống.	Tham số lọc	Danh sách truy vấn
UC13	Giám sát dịch vụ	Theo dõi trạng thái hoạt động của các dịch vụ trong hệ thống.	Không	Trạng thái dịch vụ

Bảng 3.2. Use-case cho Quản trị viên (Admin)

3.1.3. Yêu cầu phi chức năng

Mã NFR	Loại	Yêu cầu	Chỉ tiêu đo lường	Ghi chú
NFR01	Hiệu năng	Thời gian phản hồi truy vấn Q&A	≤ 15 giây cho P90	Bao gồm: embed + search + rerank + LLM
NFR02	Hiệu năng	Thời gian tìm kiếm vector	≤ 1000 ms (P95)	pgvector + HNSW
NFR03	Hiệu năng	Thời gian upload & xử lý văn bản	≤ 2 phút / file 10MB (P95)	Chunking + embedding
NFR04	Hiệu năng	Thời gian khởi động hệ thống	≤ 3 phút	docker-compose up

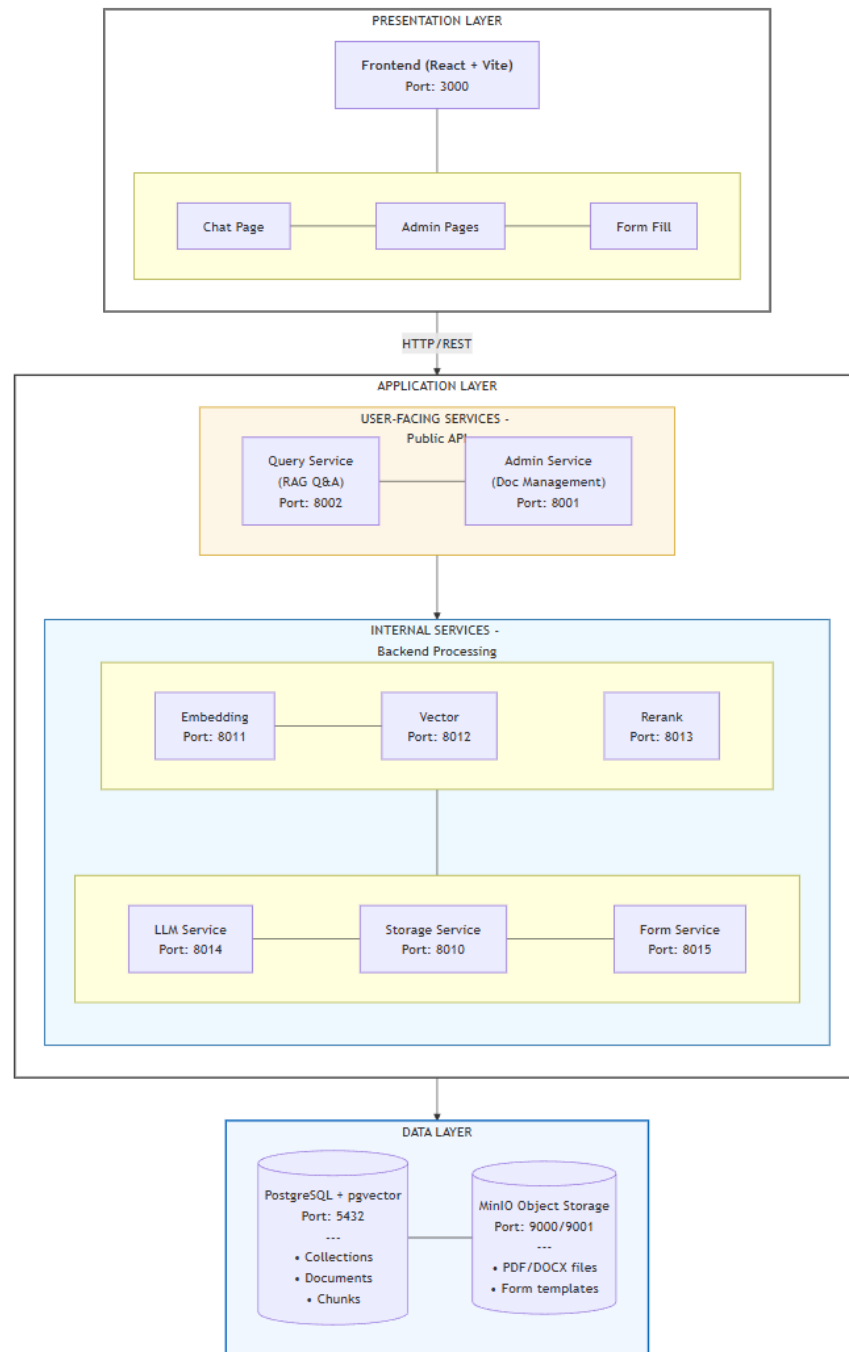
Mã NFR	Loại	Yêu cầu	Chỉ tiêu đo lường	Ghi chú
NFR05	Dung lượng	Quy mô dữ liệu hỗ trợ	≥ 1000 văn bản	Nêu theo tài nguyên máy local
NFR06	Bảo mật	Xác thực Admin	JWT expiration 2 giờ	Admin bắt buộc đăng nhập
NFR07	Bảo mật	Bảo vệ API admin	Token middleware cho /admin/*	Trừ /health và /auth/*
NFR08	Độ tin cậy	Uptime (môi trường local)	$\geq 95\%$ trong giờ sử dụng	Local machine không cam kết 99% như server
NFR09	Độ tin cậy	Data persistence	Không mất dữ liệu khi restart	PostgreSQL + MinIO volumes
NFR10	Khả năng bảo trì	Module hóa	Microservices độc lập	Dễ thay thế từng service
NFR11	Khả năng bảo trì	Container hóa	Tất cả services chạy trong Docker	docker-compose.yml
NFR12	Khả dụng	Health monitoring	100% services có /health	Giúp phát hiện lỗi nhanh
NFR13	Tính tương thích	Trình duyệt hỗ trợ	Chrome/Edge/Firefox mới	React frontend
NFR14	Ngôn ngữ	Hỗ trợ tiếng Việt	Embedding tiếng Việt	vietnamese-document-embedding

Bảng 3.3. Yêu cầu phi chức năng

3.2. Thiết kế kiến trúc tổng thể

3.2.1. Kiến trúc mức cao (High-level architecture)

Hệ thống LegalRAG được thiết kế theo kiến trúc microservices theo mô hình ba tầng (three-layer architecture), bao gồm tầng trình bày (Presentation Layer), tầng ứng dụng (Application Layer) và tầng dữ liệu (Data Layer), như minh họa trong Hình 3.2. Kiến trúc này nhằm đảm bảo tính phân tách trách nhiệm giữa các thành phần, đồng thời hỗ trợ khả năng mở rộng và bảo trì hệ thống trong quá trình vận hành.



Hình 3.2. Sơ đồ kiến trúc tổng quan

Tầng trình bày cung cấp giao diện tương tác cho người dùng và quản trị viên thông qua ứng dụng web, bao gồm các chức năng hỏi đáp pháp luật, quản trị dữ liệu và điền biểu mẫu. Tầng này tiếp nhận yêu cầu từ người dùng và chuyển tiếp đến tầng ứng dụng thông qua giao thức HTTP/REST.

Tầng ứng dụng là lõi xử lý nghiệp vụ của hệ thống, được tổ chức theo kiến trúc microservices và chia thành hai nhóm: các dịch vụ hướng người dùng và các dịch vụ xử lý nội bộ. Nhóm dịch vụ hướng người dùng (User-facing services) cung cấp

các API phục vụ truy vấn hỏi đáp và quản trị dữ liệu, trong khi các dịch vụ xử lý nội bộ đảm nhiệm các tác vụ chuyên biệt trong pipeline xử lý ngôn ngữ và dữ liệu.

Tầng dữ liệu bao gồm cơ sở dữ liệu PostgreSQL tích hợp pgvector để lưu trữ metadata và vector embedding, cùng với hệ thống lưu trữ đối tượng MinIO để quản lý các tài liệu gốc. Cách tổ chức này đảm bảo tính nhất quán dữ liệu, đồng thời hỗ trợ lưu trữ và truy xuất hiệu quả các tài liệu dung lượng lớn, tạo nền tảng cho việc triển khai một hệ thống LegalRAG có khả năng mở rộng và đáp ứng các yêu cầu về hiệu năng và độ tin cậy.

3.2.2. Thiết kế module theo microservices

Hệ thống LegalRAG được phân rã thành các microservices độc lập, mỗi dịch vụ đảm nhiệm một chức năng nghiệp vụ cụ thể trong toàn bộ pipeline xử lý, nhằm tuân thủ nguyên tắc phân tách trách nhiệm và hỗ trợ triển khai, mở rộng độc lập.

Nhóm dịch vụ	Service	Chức năng chính	Công nghệ chính	Ghi chú
Presentation	Frontend	Giao diện người dùng, quản trị và điền biểu mẫu	React + Vite	Web application
User-facing	Query Service	Điều phối pipeline hỏi đáp pháp luật	FastAPI	Orchestration
User-facing	Admin Service	Quản trị dữ liệu và thêm tài liệu	FastAPI	Orchestration
Internal	Embedding Service	Sinh vector embedding	Sentence-Transformers	NLP processing
Internal	Vector Service	Truy hồi ngữ nghĩa	PostgreSQL + pgvector	Vector search
Internal	Rerank Service	Tái xếp hạng kết quả	Cross-Encoder	Relevance scoring
Internal	LLM Service	Sinh câu trả lời	LLM (local/API)	Text generation
Internal	Storage Service	Quản lý tài liệu gốc	MinIO	Object storage
Internal	Form Service	Xử lý biểu mẫu, CCCD	python-docx	Domain-specific

Bảng 3.4. Phân rã các microservices trong hệ thống LegalRAG

Trong kiến trúc này, Query Service và Admin Service đều được thiết kế theo mô hình điều phối (orchestration), đảm nhiệm các luồng nghiệp vụ khác nhau của hệ thống. Query Service chịu trách nhiệm điều phối pipeline hỏi đáp pháp luật, bao gồm các bước truy hồi ngữ nghĩa, tái xếp hạng và sinh câu trả lời. Admin Service điều phối các luồng quản trị và ingest dữ liệu, phục vụ việc cập nhật và duy trì kho tri thức pháp luật.

Các dịch vụ xử lý nội bộ như Embedding Service, Vector Service, Rerank Service và LLM Service được tách riêng nhằm đáp ứng các yêu cầu tính toán khác nhau trong pipeline. Cách phân rã này cho phép hệ thống linh hoạt trong việc phân bổ tài nguyên và mở rộng theo nhu cầu thực tế.

Ngoài ra, Storage Service và Form Service đảm nhiệm việc quản lý tài liệu gốc và xử lý biểu mẫu, giúp tách biệt rõ ràng giữa tầng xử lý ngôn ngữ và tầng quản lý dữ liệu, phù hợp với đặc thù của hệ thống LegalRAG trong lĩnh vực pháp luật.

3.2.3. Cấu hình mạng và Port Mapping

Hệ thống LegalRAG được triển khai theo kiến trúc microservices và sử dụng Docker để đóng gói, triển khai các dịch vụ. Tất cả các service trong hệ thống được kết nối thông qua một Docker bridge network duy nhất, cho phép các container giao tiếp nội bộ với nhau thông qua tên service, đồng thời hạn chế việc phơi bày các dịch vụ xử lý nội bộ ra bên ngoài.

Trong kiến trúc này, các service được phân chia theo phạm vi truy cập mạng, bao gồm các service hướng người dùng (external-facing services) và các service xử lý nội bộ (internal services). Các service hướng người dùng cung cấp API công khai cho frontend và người dùng cuối, trong khi các service xử lý nội bộ chỉ được phép truy cập trong mạng nội bộ của hệ thống nhằm tăng cường tính bảo mật và kiểm soát luồng giao tiếp.

Nhóm service	Dải port	Phạm vi truy cập
Frontend	3000	Truy cập từ trình duyệt
User-facing services	8001 – 8002	API công khai
Internal services	8010 – 8015	Chỉ truy cập nội bộ
Infrastructure	5432, 9000–9001	Nội bộ và phục vụ quản trị/debug

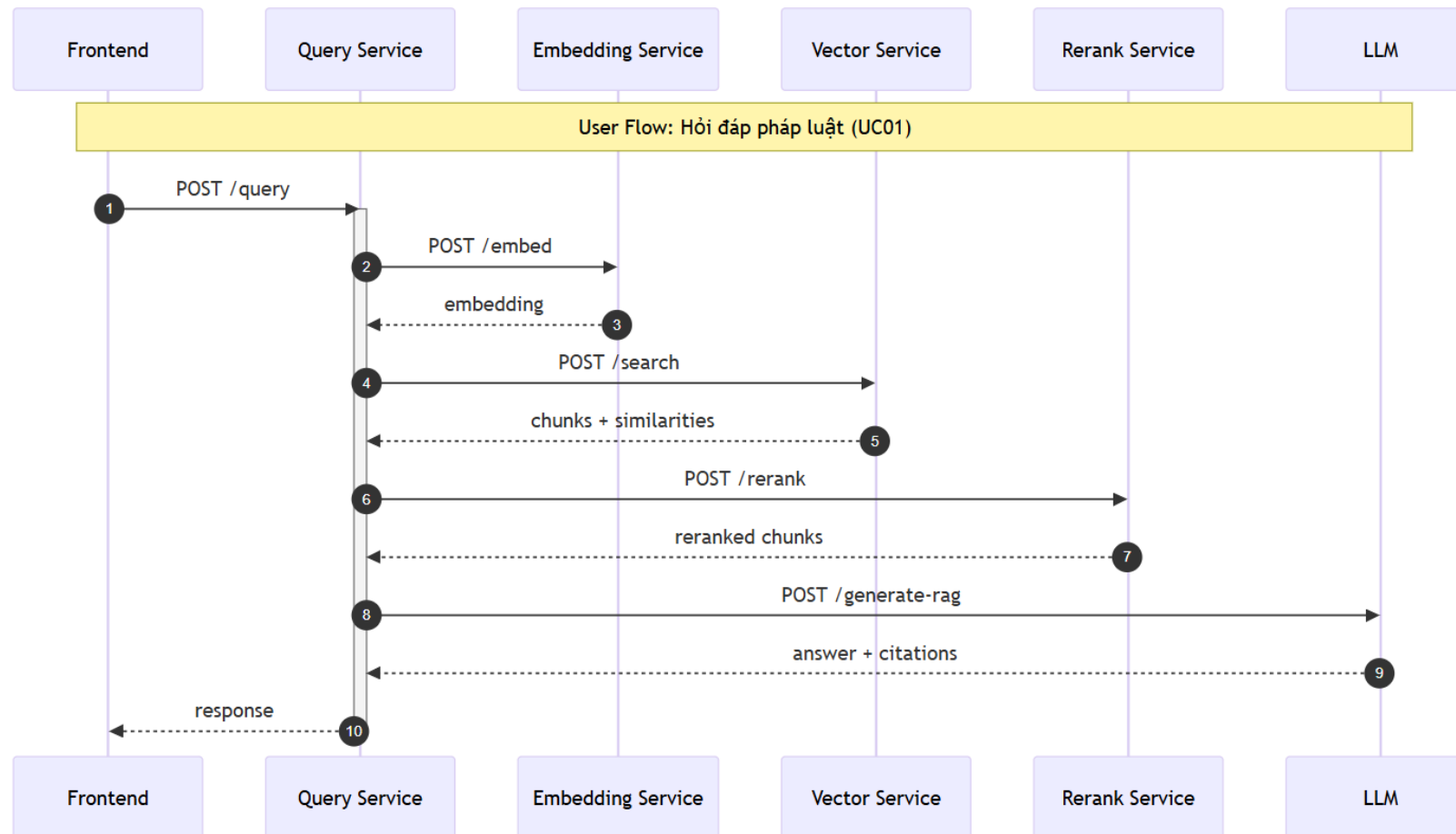
Bảng 3.5. Phân nhóm service theo cấu hình port và phạm vi truy cập

Việc quy ước dải port theo nhóm chức năng giúp hệ thống dễ dàng quản lý, mở rộng và giám sát trong quá trình vận hành. Cách tiếp cận này cũng hỗ trợ việc tách biệt rõ ràng giữa các thành phần giao tiếp với người dùng và các thành phần xử lý nội bộ, phù hợp với yêu cầu bảo mật và khả năng mở rộng của hệ thống LegalRAG.

Về mặt phụ thuộc giữa các service, frontend chỉ giao tiếp trực tiếp với các service hướng người dùng như Query Service và Admin Service. Các service này đóng vai trò điều phối, lần lượt gọi đến các service xử lý nội bộ như Embedding Service, Vector Service, Rerank Service, LLM Service và các thành phần lưu trữ dữ liệu. Cách tổ chức này đảm bảo luồng xử lý rõ ràng, tránh phụ thuộc chéo không cần thiết giữa các service và góp phần nâng cao khả năng bảo trì hệ thống.

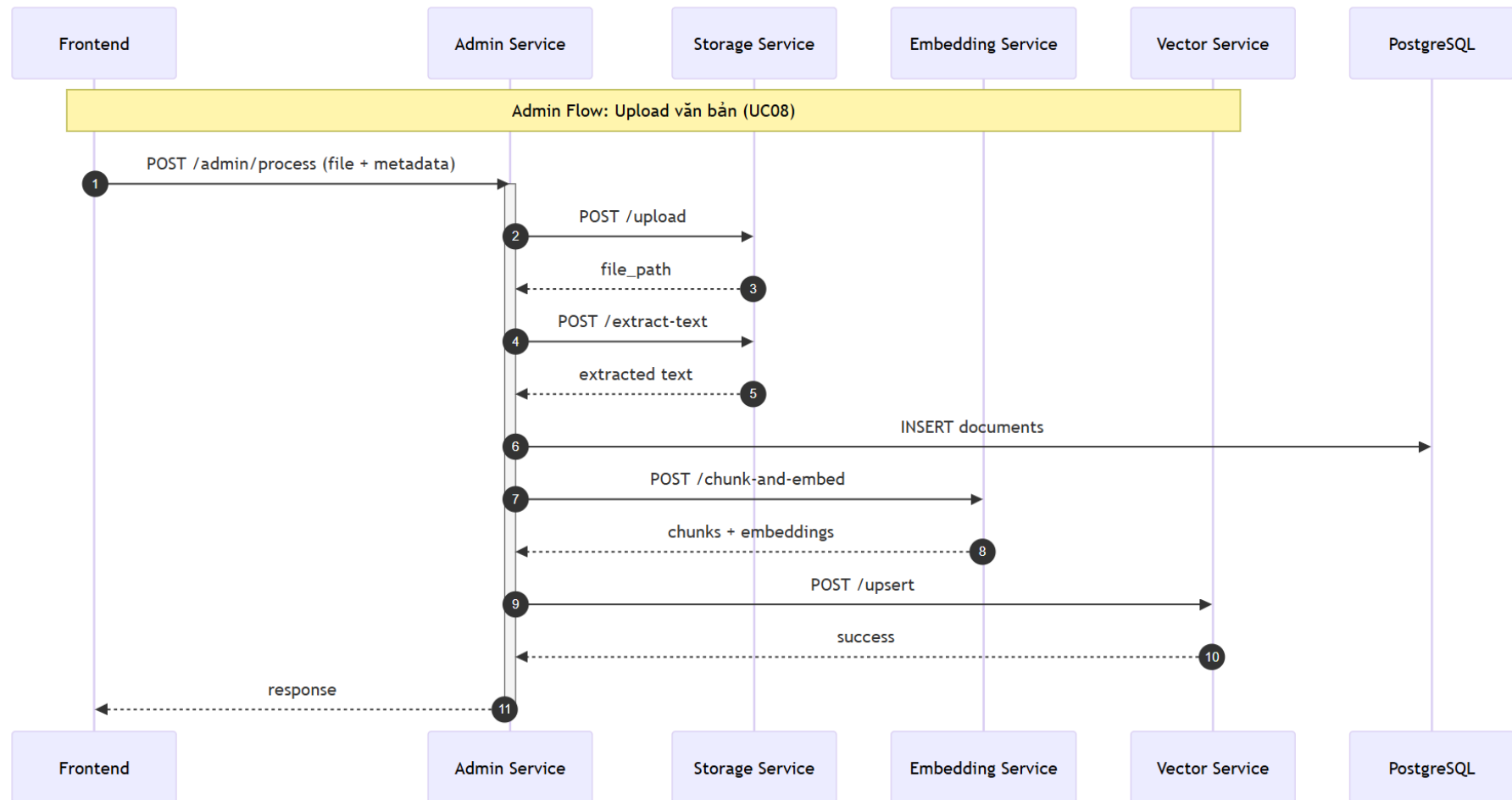
3.2.4. Luồng giao tiếp

User Flow: Hỏi đáp pháp luật (UC01)



Hình 3.3. User Flow – UC01

Admin Flow: Upload văn bản (UC08)



Hình 3.4. Admin Flow – UC08

Hình 3.3 và Hình 3.4 lần lượt minh họa luồng giao tiếp cho chức năng hỏi đáp pháp luật và luồng thêm dữ liệu trong hệ thống LegalRAG. Đối với luồng hỏi đáp, Query Service đóng vai trò điều phối, lần lượt gọi các dịch vụ xử lý nội bộ để sinh embedding, truy hồi ngữ nghĩa, tái xếp hạng và sinh câu trả lời. Trong luồng quản trị, Admin Service điều phối quá trình lưu trữ tài liệu gốc, trích xuất nội dung, sinh embedding và cập nhật dữ liệu vào hệ thống. Cách tổ chức luồng giao tiếp theo mô hình điều phối giúp giảm phụ thuộc chéo giữa các service và phù hợp với kiến trúc microservices đã đề xuất.

3.2.5. Thiết kế API & giao tiếp liên service

Hệ thống LegalRAG sử dụng thiết kế API theo HTTP/REST kết hợp với các action endpoints, nhằm phù hợp với đặc thù của hệ thống microservices xử lý AI. Các thao tác quản lý dữ liệu tuân thủ nguyên tắc RESTful, trong khi các nghiệp vụ xử lý phức tạp được thiết kế theo hướng action-based để phản ánh đúng bản chất điều phối pipeline của hệ thống.

Các nguyên tắc thiết kế API chính bao gồm: sử dụng HTTP methods chuẩn, trao đổi dữ liệu dưới định dạng JSON, thiết kế stateless và cung cấp endpoint kiểm tra trạng thái cho mỗi service.

Trong hệ thống, các endpoint RESTful được áp dụng cho các thao tác CRUD đối với tài nguyên dữ liệu như collections và metadata. Ngược lại, các nghiệp vụ như truy vấn hỏi đáp, xử lý thêm tài liệu và sinh câu trả lời được triển khai thông qua các action endpoints do bản chất của các thao tác này là các pipeline xử lý nhiều bước, không đại diện cho một tài nguyên độc lập.

Các service giao tiếp với nhau thông qua HTTP/REST trong mạng nội bộ Docker. Mỗi lời gọi liên service được cấu hình timeout phù hợp với đặc tính xử lý của từng thành phần nhằm tránh tình trạng treo hệ thống khi xảy ra độ trễ cao ở các service xử lý nặng như reranking hoặc sinh văn bản.

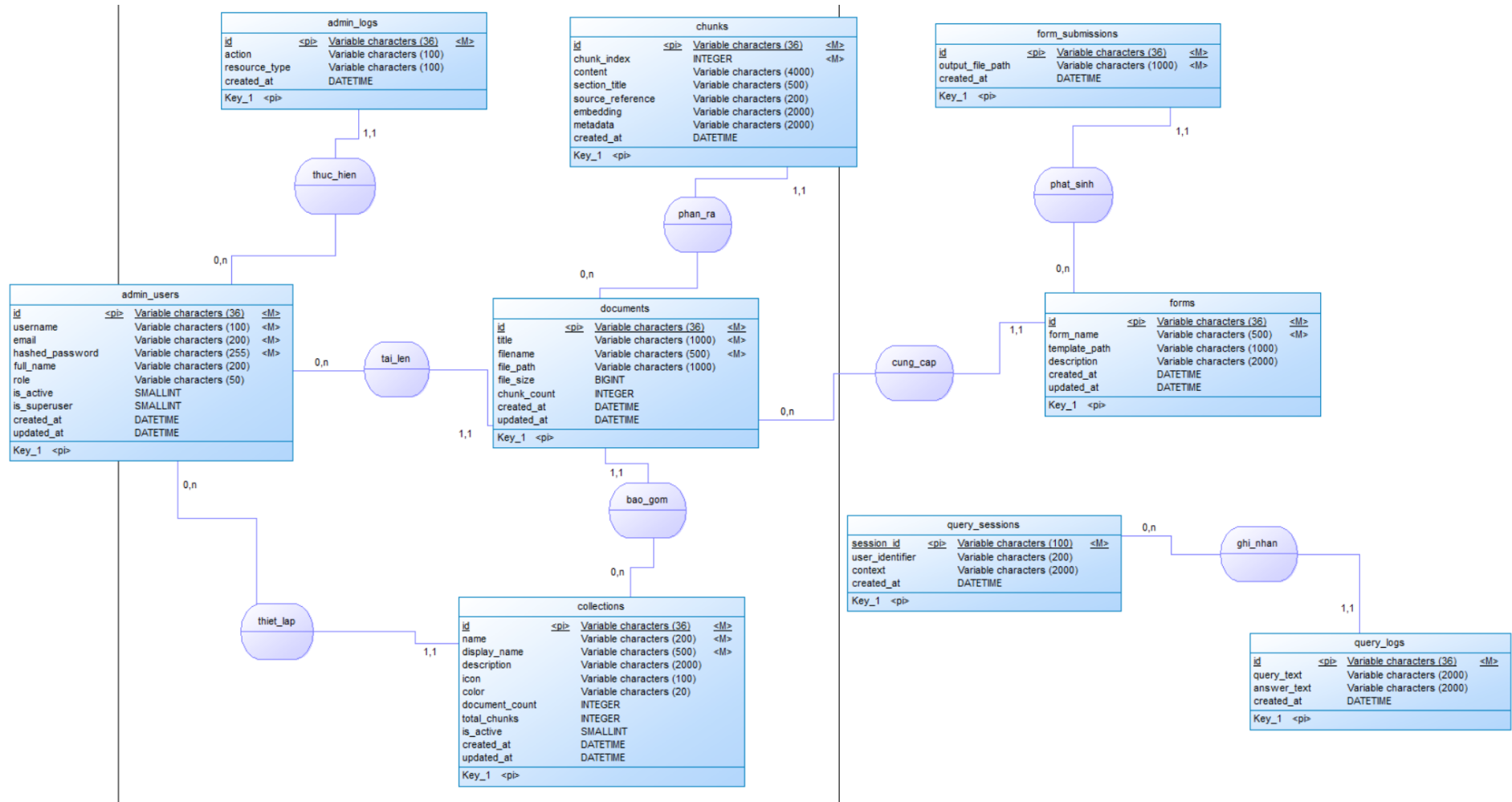
Hệ thống áp dụng cơ chế health check thông qua endpoint /health cho mỗi service nhằm hỗ trợ giám sát và phát hiện sớm lỗi trong quá trình vận hành. Các endpoint quản trị được bảo vệ bằng cơ chế xác thực JWT, trong khi các endpoint phục vụ người dùng cuối được công khai để đảm bảo tính thuận tiện khi sử dụng.

Việc xử lý lỗi được thực hiện thống nhất thông qua các HTTP status code chuẩn, cho phép service gọi nhận biết và phản ứng phù hợp khi xảy ra lỗi phụ thuộc hoặc timeout.

3.3. Thiết kế dữ liệu

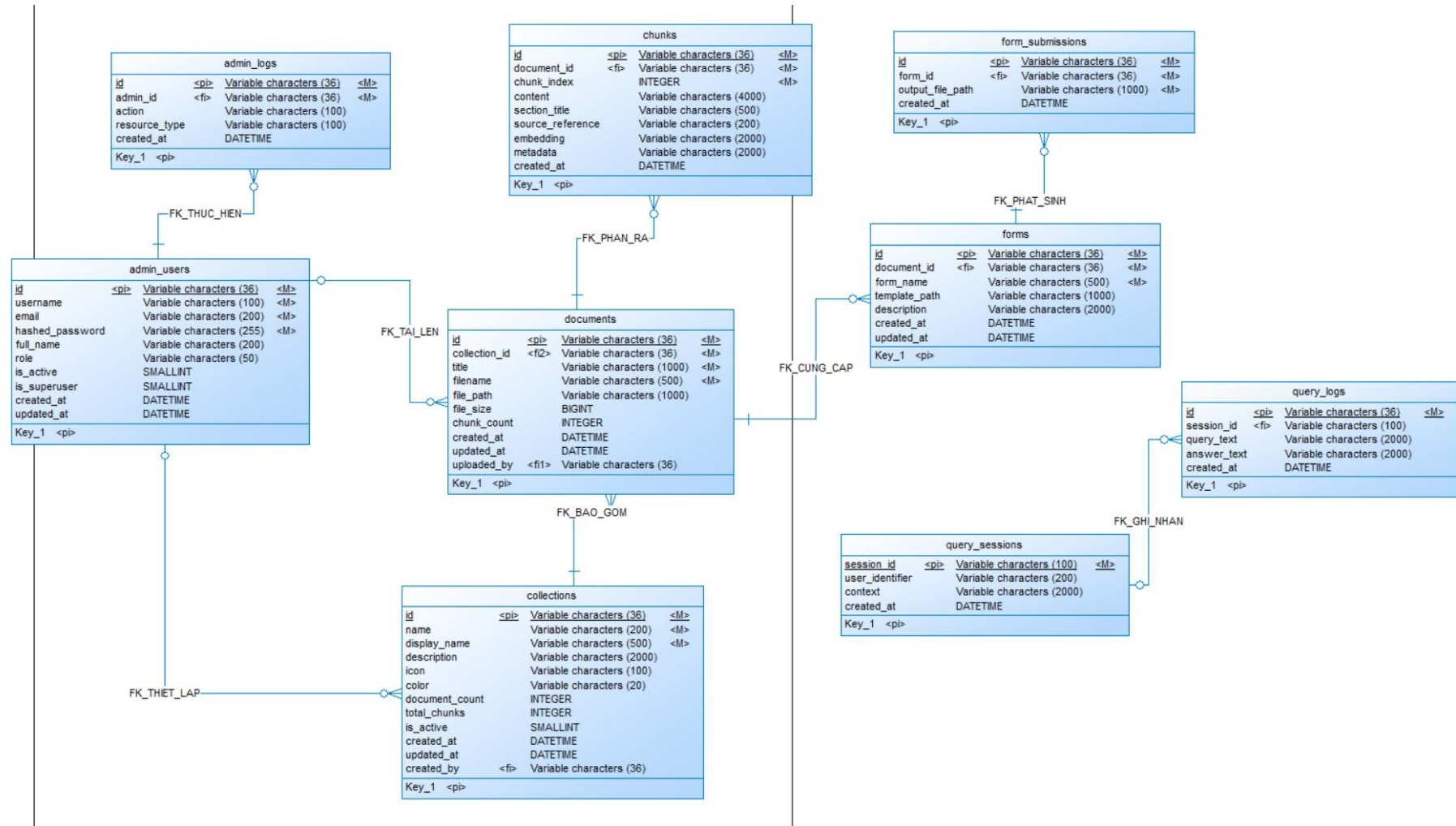
Mục này trình bày thiết kế dữ liệu của hệ thống LegalRAG thông qua ba mức mô hình hóa gồm mô hình dữ liệu tổng thể, mô hình dữ liệu logic và mô hình dữ liệu vật lý. Các mô hình được xây dựng theo hướng từ khái niệm đến triển khai nhằm làm rõ cấu trúc dữ liệu, mối quan hệ giữa các thực thể và cách tổ chức dữ liệu phục vụ cho pipeline Retrieval-Augmented Generation.

3.3.1. Mô hình mức quan niệm



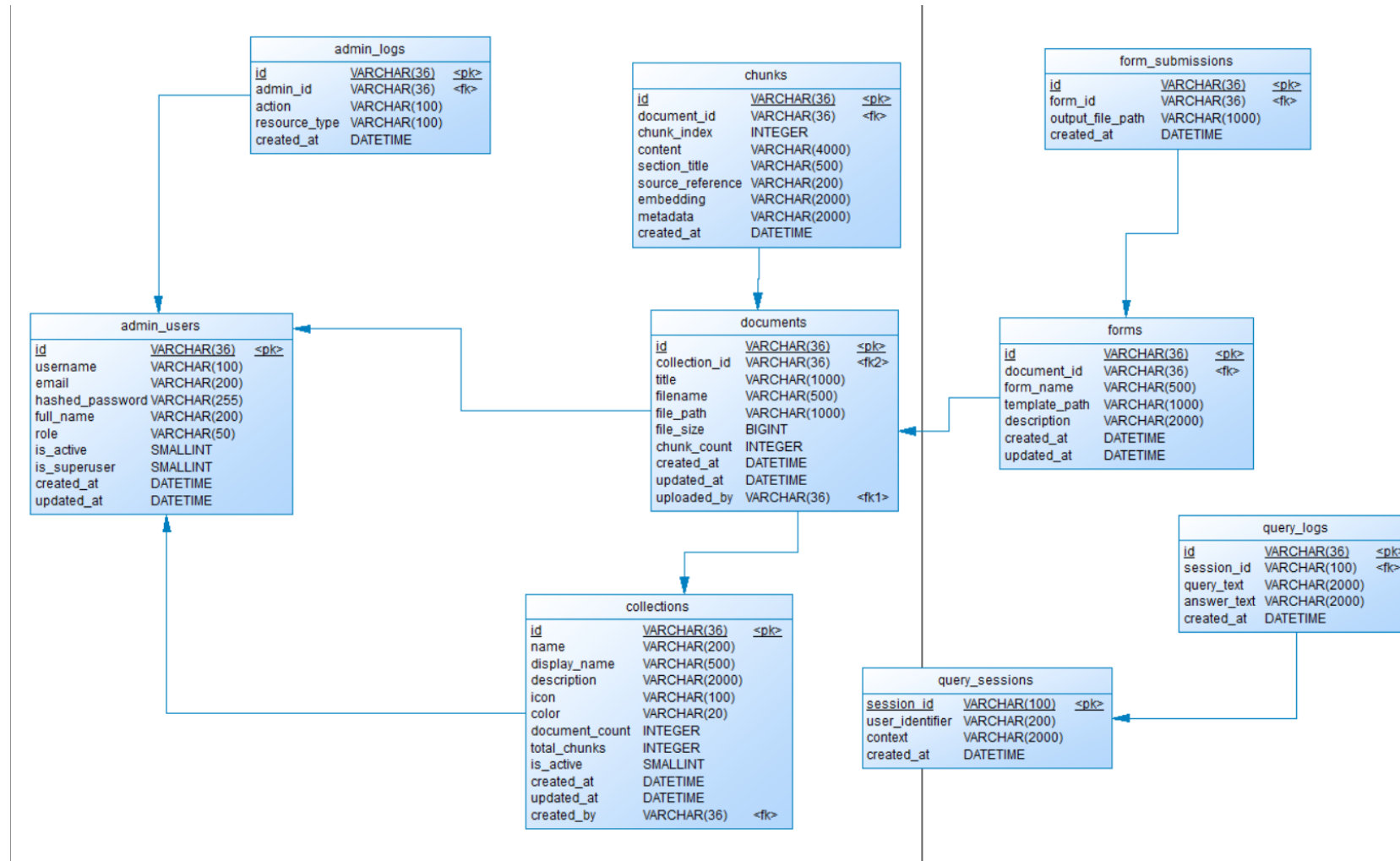
Hình 3.5. Mô hình mức quan niệm

3.3.2. Mô hình mức luận lý



Hình 3.6. Mô hình mức luận lý

3.3.3. Mô hình mức vật lý

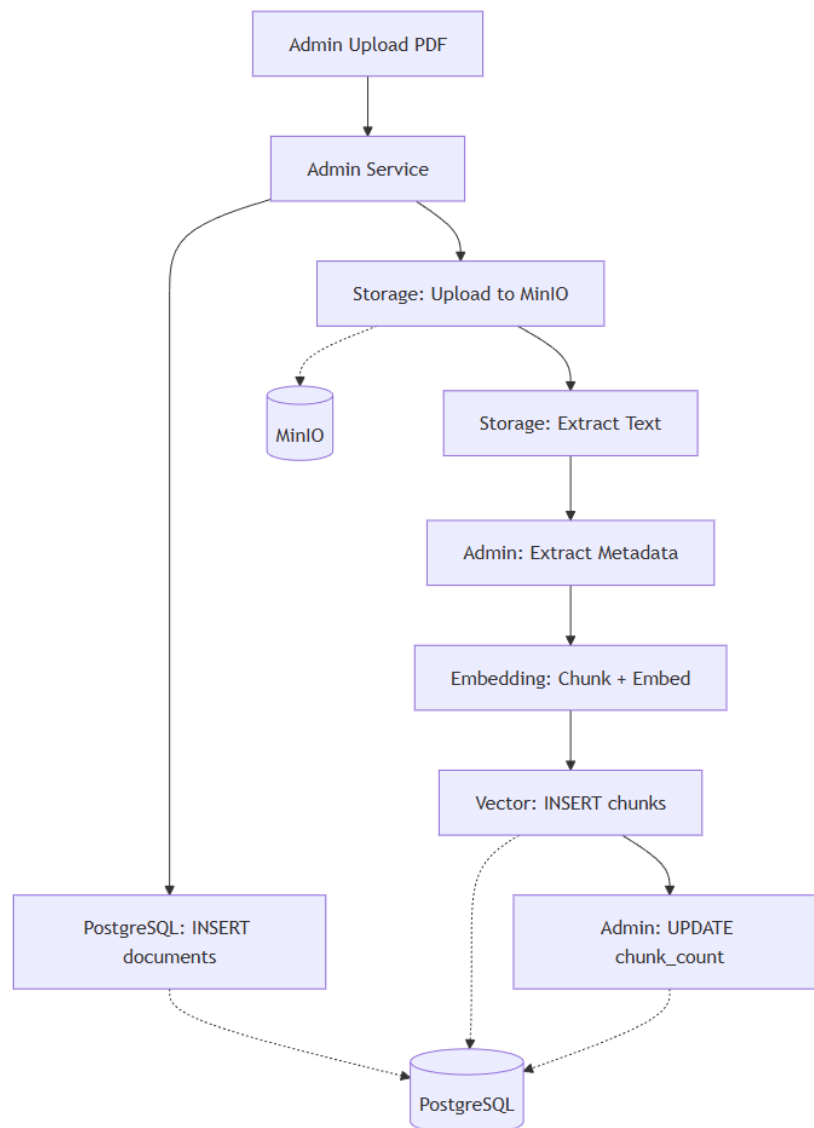


Hình 3.7. Mô hình mức vật lý

3.4. Thiết kế pipeline RAG

3.4.1. Document ingestion flow (upload → Knowledge Base)

Quy trình document ingestion mô tả chuỗi các bước xử lý dữ liệu khi một tài liệu mới được đưa vào hệ thống LegalRAG. Sau khi quản trị viên tải tài liệu lên, file gốc được lưu trữ tại hệ thống lưu trữ đối tượng MinIO nhằm đảm bảo khả năng truy xuất và đối chiếu nguồn. Nội dung văn bản được trích xuất từ tài liệu, sau đó được chuẩn hóa và tách thành các đoạn (chunks). Mỗi đoạn văn bản sau đó được sinh embedding và lưu trữ cùng metadata liên quan trong cơ sở dữ liệu PostgreSQL tích hợp pgvector. Toàn bộ dữ liệu sau khi xử lý tạo thành Knowledge Base, phục vụ cho quá trình truy hỏi ngữ nghĩa và trích dẫn nguồn trong pipeline hỏi đáp pháp luật.



Hình 3.8. Document ingestion flow (Upload → Knowledge Base)

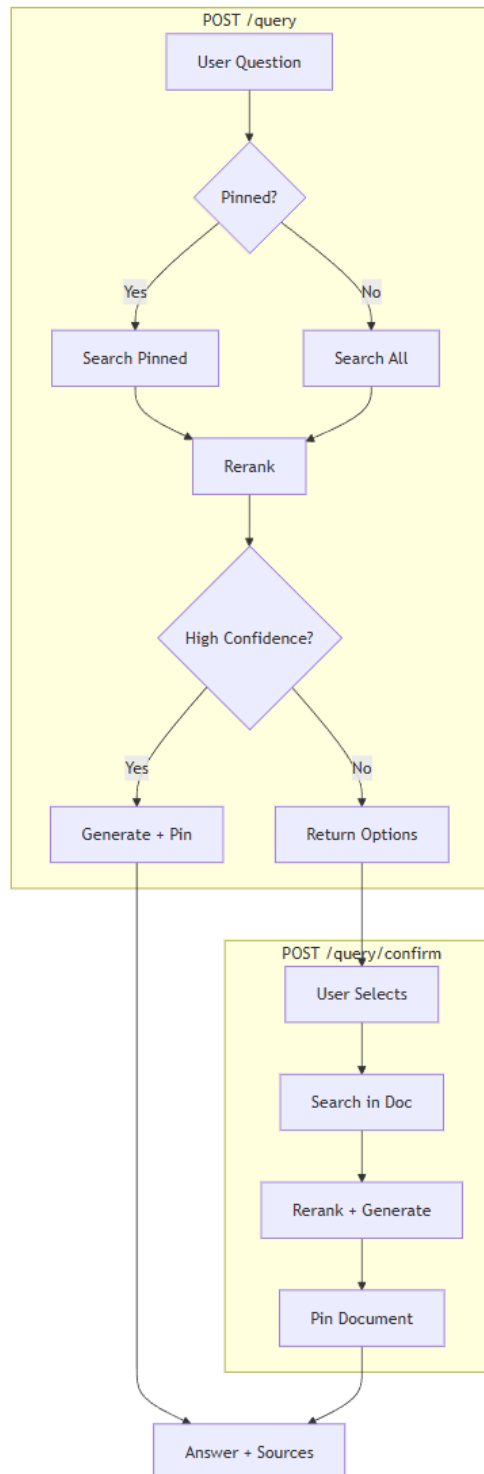
3.4.2. Admin flow

Trong hệ thống LegalRAG, luồng quản trị viên không hình thành một pipeline xử lý độc lập mà chủ yếu đóng vai trò khởi tạo và kiểm soát quá trình document ingestion. Mọi thao tác của quản trị viên, bao gồm tải mới, thay thế hoặc cập nhật tài liệu, đều dẫn đến việc kích hoạt pipeline ingestion đã được mô tả ở mục 3.4.1. Do đó, Admin flow được xem như một góc nhìn nghiệp vụ của pipeline ingestion, tập trung vào việc quản lý vòng đời dữ liệu và đảm bảo tính nhất quán của Knowledge Base.

Bên cạnh việc khởi tạo ingestion, giao diện quản trị còn cung cấp các chỉ số tổng quan ở mức đơn giản như số lượng tài liệu, số lượng đoạn văn bản (chunks) và trạng thái xử lý. Các thông tin này hỗ trợ quản trị viên theo dõi tình trạng Knowledge Base mà không can thiệp trực tiếp vào pipeline xử lý dữ liệu. Chi tiết luồng giao tiếp giữa các service trong trường hợp quản trị viên tải văn bản (UC08) đã được trình bày tại mục 3.2.4, Hình 3.4.

3.4.3. Query flow

Query flow mô tả pipeline xử lý câu hỏi của người dùng theo mô hình Retrieval-Augmented Generation, có kết hợp cơ chế duy trì ngữ cảnh hội thoại. Quy trình bắt đầu từ việc tiếp nhận câu hỏi, thực hiện truy hồi ngữ nghĩa trên Knowledge Base, tái xếp hạng kết quả và sinh câu trả lời kèm trích dẫn. Trong trường hợp kết quả truy hồi chưa đủ rõ ràng, hệ thống yêu cầu người dùng xác nhận văn bản nguồn trước khi sinh câu trả lời cuối cùng.

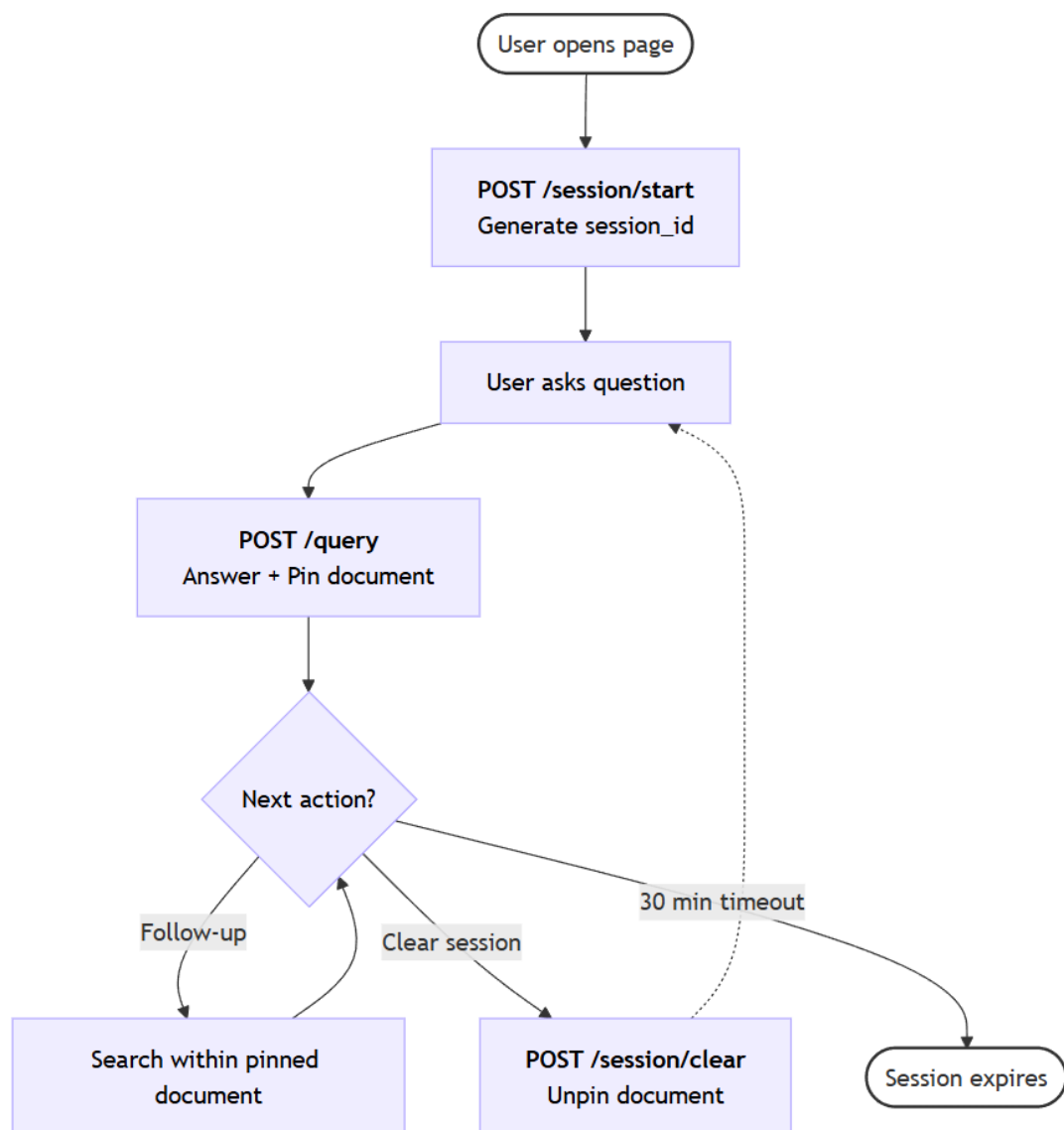


Hình 3.9. Luồng xử lý Query flow

Hình 3.9 minh họa pipeline xử lý Query flow với các nhánh quyết định liên quan đến document pinning và cơ chế xác nhận văn bản nguồn. Chi tiết luồng giao tiếp giữa các service trong trường hợp hỏi đáp pháp luật (UC01) đã được trình bày tại mục 3.2.4, Hình 3.3.

3.4.4. Cơ chế cải thiện hội thoại

Để đảm bảo tính nhất quán trong quá trình hội thoại, hệ thống LegalRAG áp dụng cơ chế document pinning và nhận diện câu hỏi tiếp nối (follow-up detection). Khi người dùng đặt câu hỏi ban đầu và hệ thống xác định được văn bản pháp luật phù hợp với độ tin cậy cao, văn bản này sẽ được ghim (pinned) vào phiên hội thoại. Các câu hỏi tiếp theo của người dùng sẽ ưu tiên truy hồi thông tin trong phạm vi văn bản đã ghim thay vì toàn bộ Knowledge Base, từ đó giảm nhiễu và tăng độ chính xác của câu trả lời.



Hình 3.10. Vòng đời phiên hội thoại trong hệ thống

Như minh họa trong Hình 3.10, mỗi phiên hội thoại được khởi tạo khi người dùng bắt đầu tương tác với hệ thống và được duy trì xuyên suốt các lượt hỏi đáp liên

tiếp. Trong suốt vòng đời của phiên, văn bản pháp luật đã được ghim đóng vai trò là ngữ cảnh chính để xử lý các câu hỏi tiếp nối, giúp hệ thống trả lời nhất quán mà không cần truy hồi lại toàn bộ Knowledge Base.

Bên cạnh đó, hệ thống có khả năng phát hiện các câu hỏi mang tính phụ thuộc ngữ cảnh dựa trên độ dài và cấu trúc ngôn ngữ. Trong trường hợp câu hỏi quá ngắn hoặc thiếu thông tin, truy vấn sẽ được mở rộng bằng cách kết hợp nội dung câu hỏi với tiêu đề văn bản đã được ghim. Phiên hội thoại sẽ tự động hết hạn sau một khoảng thời gian không hoạt động nhằm giải phóng tài nguyên và đảm bảo hiệu quả vận hành của hệ thống.

3.4.5. Chính sách sinh câu trả lời

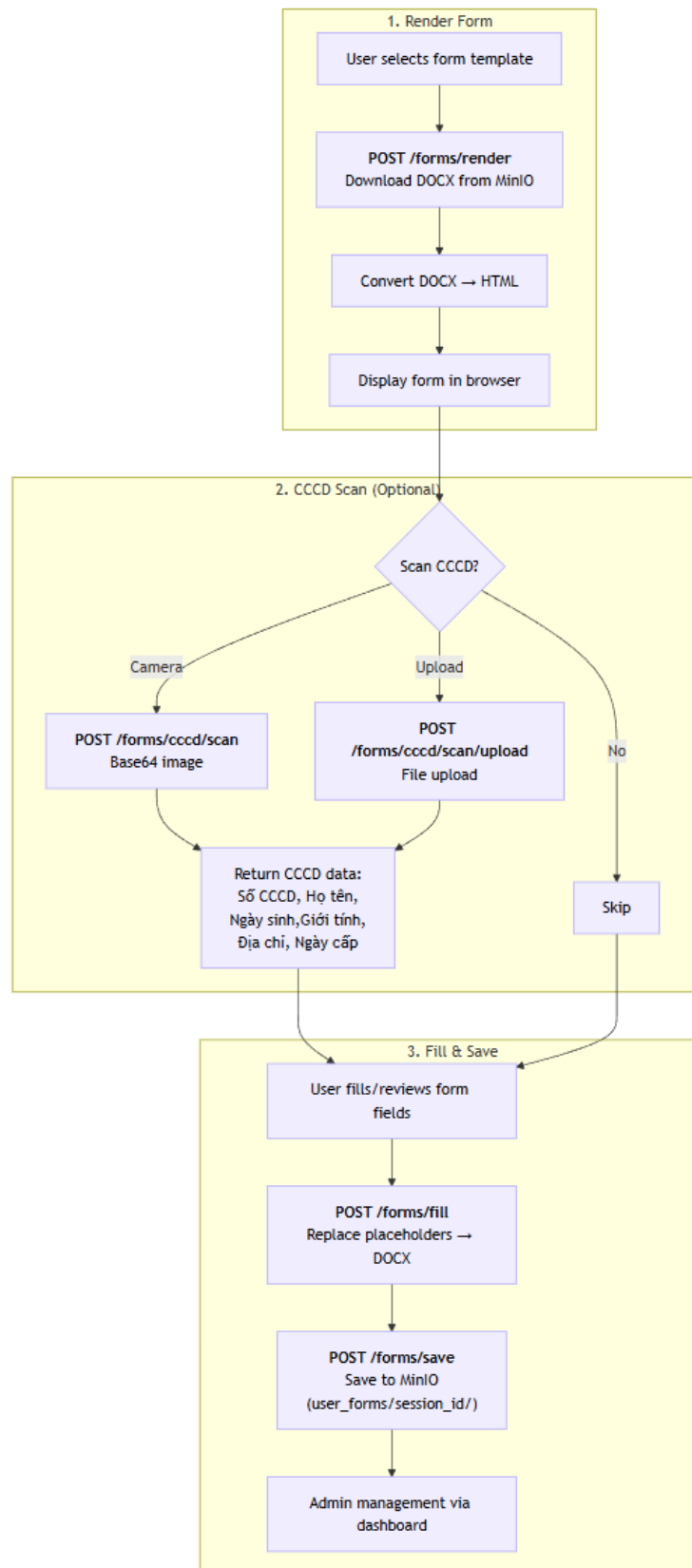
Trong hệ thống hỏi đáp pháp luật, việc sinh câu trả lời không chỉ yêu cầu tính tự nhiên về ngôn ngữ mà còn đòi hỏi độ chính xác và khả năng kiểm chứng cao. Do đó, hệ thống LegalRAG áp dụng chính sách sinh câu trả lời dựa trên nguyên tắc ràng buộc chặt chẽ vào ngữ cảnh văn bản pháp luật đã được truy hồi.

Cụ thể, mô hình ngôn ngữ lớn chỉ được phép trích xuất và diễn giải thông tin từ các đoạn văn bản đã được lựa chọn trong bước truy hồi và tái xếp hạng. Hệ thống không cho phép mô hình bổ sung thông tin từ kiến thức bên ngoài hoặc suy đoán nội dung không được đề cập trong văn bản nguồn. Trong trường hợp văn bản không chứa thông tin phù hợp với câu hỏi, hệ thống trả về thông báo tương ứng thay vì sinh câu trả lời mang tính phỏng đoán.

Ngữ cảnh cung cấp cho mô hình ngôn ngữ được xây dựng từ các đoạn văn bản có độ liên quan cao nhất, mỗi đoạn đều gắn kèm thông tin trích dẫn để đảm bảo khả năng đối chiếu nguồn. Cách tiếp cận này giúp cân bằng giữa khả năng sinh ngôn ngữ tự nhiên của mô hình và yêu cầu chính xác, minh bạch trong lĩnh vực pháp luật.

3.4.6. Pipeline điền biểu mẫu & quét CCCD

Bên cạnh chức năng hỏi đáp pháp luật, hệ thống LegalRAG tích hợp pipeline hỗ trợ điền biểu mẫu hành chính nhằm nâng cao khả năng ứng dụng trong thực tế. Pipeline này cho phép người dùng chuyển liền mạch từ giai đoạn tra cứu thông tin sang thực hiện thủ tục hành chính, đảm bảo sự nhất quán với nội dung tư vấn đã được cung cấp bởi hệ thống.



Hình 3.11. Pipeline hỗ trợ điền biểu mẫu hành chính

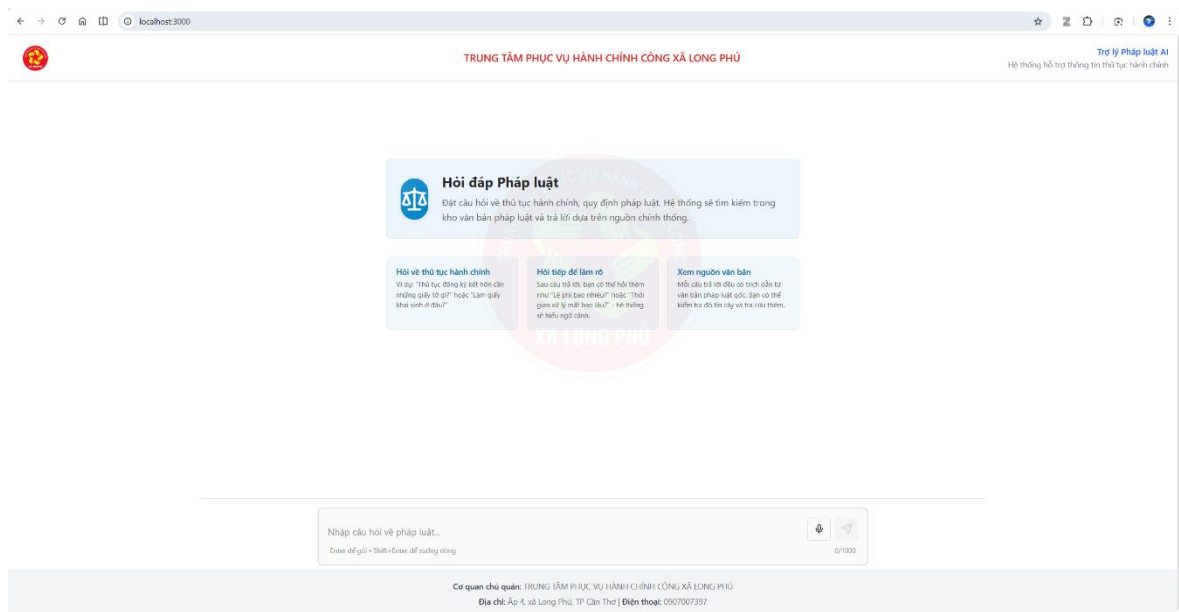
Trong pipeline này, hệ thống hỗ trợ trích xuất các thông tin cá nhân cơ bản từ căn cước công dân (CCCD) do người dùng cung cấp và sử dụng các thông tin này để

tự động điền vào biểu mẫu hành chính điện tử. Các biểu mẫu được xây dựng từ các mẫu chuẩn, trong đó các vị trí cần điền được định nghĩa sẵn theo một quy ước thống nhất. Sau khi người dùng kiểm tra và xác nhận dữ liệu, hệ thống sinh ra biểu mẫu hoàn chỉnh và lưu trữ để phục vụ tải xuống hoặc sử dụng trong phiên làm việc. Cách tiếp cận này giúp giảm thao tác nhập liệu thủ công, đồng thời tăng độ chính xác và tính tiện lợi khi thực hiện các thủ tục hành chính.

3.5. Thiết kế giao diện

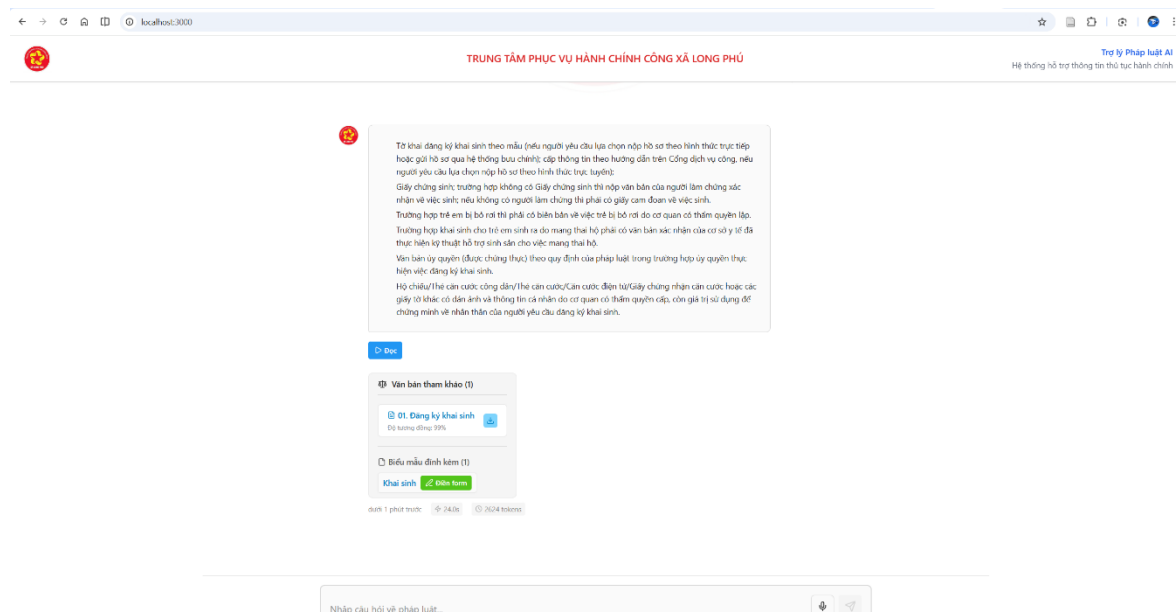
3.5.1. UI/UX user chat

Giao diện hỏi đáp pháp luật được thiết kế theo hướng tối giản, tập trung vào trải nghiệm người dùng phổ thông khi tra cứu thủ tục hành chính. Người dùng có thể đặt câu hỏi tự nhiên bằng tiếng Việt và nhận câu trả lời dựa trên văn bản pháp luật chính thống, kèm theo nguồn trích dẫn và biểu mẫu liên quan khi cần thiết.



Hình 3.12. Giao diện trang hỏi đáp pháp luật

Hình 3.12 minh họa giao diện trang hỏi đáp pháp luật của hệ thống. Giao diện cung cấp ô nhập câu hỏi trung tâm, kèm theo các gợi ý sử dụng nhằm hỗ trợ người dùng đặt câu hỏi đúng ngữ cảnh. Thiết kế này giúp giảm rào cản tiếp cận đối với người dân không có kiến thức chuyên môn về pháp luật.

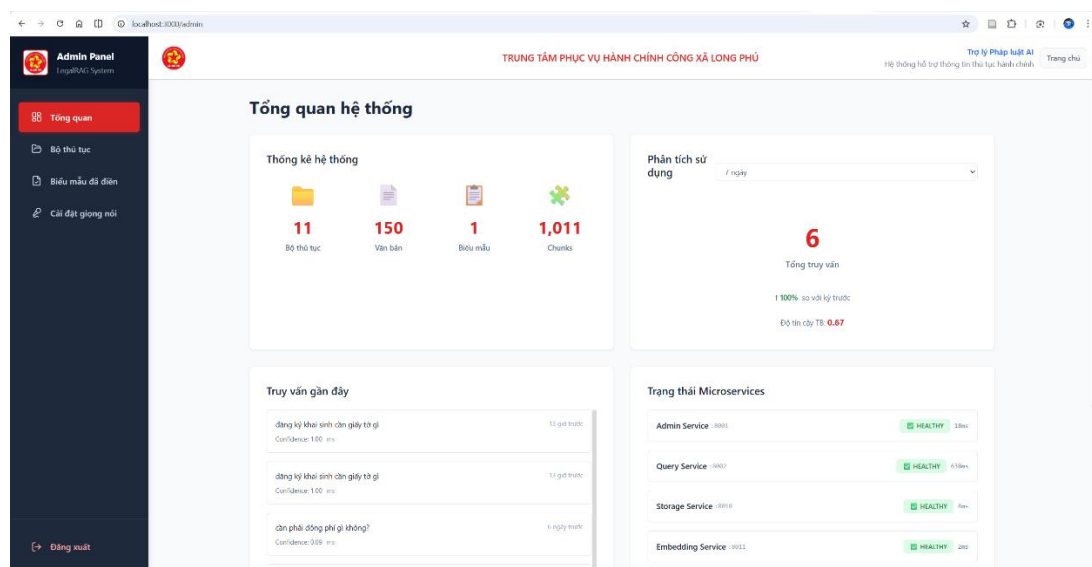


Hình 3.13. Giao diện hiển thị kết quả trả lời và nguồn tham chiếu

Hình 3.13 thể hiện giao diện hiển thị kết quả trả lời. Câu trả lời được trình bày rõ ràng, đồng thời hệ thống hiển thị danh sách văn bản pháp luật tham chiếu và mức độ liên quan. Ngoài ra, đối với các thủ tục có biểu mẫu kèm theo, hệ thống cung cấp chức năng chuyển tiếp sang pipeline điền biểu mẫu đã trình bày ở mục 3.4.6.

3.5.2. Admin dashboard

Giao diện quản trị viên cung cấp các chức năng cơ bản phục vụ quản lý Knowledge Base, bao gồm danh sách tài liệu, số lượng đoạn văn bản (chunks) và trạng thái xử lý. Giao diện này cho phép quản trị viên theo dõi tình trạng dữ liệu mà không can thiệp trực tiếp vào pipeline xử lý, phù hợp với vai trò điều phối đã mô tả.



Hình 3.14. Giao diện Admin Dashboard

3.5.3. Form fill page

Giao diện điền biểu mẫu cho phép người dùng xem và chỉnh sửa biểu mẫu hành chính trực tiếp trên trình duyệt. Biểu mẫu được sinh tự động từ các mẫu chuẩn và hiển thị dưới dạng các trường có thể điền.

Hình 3.15. Giao diện điền biểu mẫu trước khi thực hiện quét CCCD

Hình 3.16. Giao diện sau khi hệ thống quét CCCD thành công và điền form

Thiết kế giao diện này giúp giảm đáng kể thao tác nhập liệu thủ công, đồng thời đảm bảo tính nhất quán giữa thông tin tư vấn và dữ liệu được sử dụng trong biểu mẫu hành chính.

3.6. Triển khai và vận hành

Hệ thống LegalRAG được triển khai theo kiến trúc microservices, với các dịch vụ được đóng gói dưới dạng container và giao tiếp thông qua các API HTTP/REST trong mạng nội bộ.

Dữ liệu văn bản và biểu mẫu được lưu trữ trong hệ thống lưu trữ đối tượng MinIO, trong khi dữ liệu embedding và metadata được quản lý bởi PostgreSQL tích hợp pgvector. Việc tách riêng tầng lưu trữ và tầng xử lý giúp hệ thống dễ dàng mở rộng, bảo trì và nâng cấp trong tương lai.

Trong quá trình vận hành, hệ thống hỗ trợ giám sát trạng thái các service thông qua các endpoint health check và theo dõi các chỉ số cơ bản như số lượng truy vấn, tài liệu và biểu mẫu. Cách tiếp cận này đảm bảo hệ thống hoạt động ổn định và đáp ứng yêu cầu sử dụng trong môi trường thực tế.

3.7. Kết luận chương

Chương này đã trình bày chi tiết quá trình hiện thực hóa hệ thống LegalRAG, bao gồm thiết kế pipeline RAG, luồng quản trị và luồng truy vấn, cơ chế cải thiện hội thoại, cũng như tích hợp chức năng điền biểu mẫu hành chính. Các thành phần của hệ thống được thiết kế theo kiến trúc microservices nhằm đảm bảo tính mở rộng, linh hoạt và phù hợp với đặc thù của bài toán hỏi đáp pháp luật.

Kết quả triển khai cho thấy hệ thống không chỉ hỗ trợ tra cứu thông tin pháp luật mà còn góp phần rút ngắn quy trình thực hiện thủ tục hành chính, tạo tiền đề cho việc ứng dụng các kỹ thuật trí tuệ nhân tạo trong hỗ trợ tra cứu và thực hiện thủ tục hành chính. Chương tiếp theo sẽ trình bày kết quả thực nghiệm và đánh giá hiệu năng của hệ thống.

CHƯƠNG 4 KẾT QUẢ NGHIÊN CỨU

4.1. Môi trường thực nghiệm

4.1.1. Cấu hình phần cứng và phần mềm

Môi trường thực nghiệm được thiết lập nhằm phản ánh điều kiện triển khai thực tế của một hệ thống hỏi đáp pháp luật quy mô nhỏ đến trung bình. Các thí nghiệm được thực hiện trên một máy tính cá nhân với cấu hình phần cứng và phần mềm như trình bày trong Bảng 4.1.

Thành phần	Thông số
CPU	Intel Core i7-12700H (14 cores, 20 threads)
RAM	32 GB DDR5
GPU	NVIDIA GeForce RTX 3060 Laptop (6 GB VRAM)
Storage	SSD NVMe 1 TB

Bảng 4.1. Thông số hệ thống

Cấu hình này phản ánh điều kiện triển khai thực tế của hệ thống LegalRAG trong môi trường cục bộ (local deployment) với tài nguyên phần cứng hạn chế.

4.1.2. Dữ liệu và tập câu hỏi đánh giá

a) Dữ liệu văn bản pháp luật

Nghiên cứu sử dụng bộ dữ liệu thủ tục hành chính công được thu thập từ Cổng Dịch vụ công Quốc gia. Các văn bản được lựa chọn thuộc các lĩnh vực phổ biến, phản ánh nhu cầu tra cứu thủ tục hành chính thường gặp trong thực tế.

Tập dữ liệu bao gồm 15 văn bản pháp luật, được phân chia theo bốn lĩnh vực chính là hộ tịch, đăng ký kinh doanh, nuôi con nuôi và bảo hiểm y tế. Sau bước tiền xử lý, các văn bản được chia thành các đoạn nội dung nhỏ (chunks) nhằm phục vụ cho quá trình tạo vector embedding và truy hồi ngữ nghĩa. Thống kê chi tiết về số lượng văn bản và số đoạn nội dung của từng lĩnh vực được trình bày trong Bảng 4.2.

Lĩnh vực	Số văn bản	Số chunks	Mô tả
Hộ tịch	7	89	Khai sinh, kết hôn, khai tử, giám hộ
Doanh nghiệp	4	34	Đăng ký hộ kinh doanh
Nuôi con nuôi	2	13	Đăng ký nuôi con nuôi trong nước
Bảo hiểm y tế	2	11	Cấp thẻ BHYT, khám chữa bệnh
Tổng	15	135	

Bảng 4.2. Thống kê dữ liệu văn bản

Việc phân chia dữ liệu theo lĩnh vực giúp đảm bảo tính đa dạng của nội dung pháp lý, đồng thời tạo điều kiện thuận lợi cho việc đánh giá hiệu quả truy hồi của hệ thống trên các nhóm thủ tục khác nhau.

b) Tập câu hỏi đánh giá (Test Set)

Trên cơ sở tập dữ liệu văn bản, một tập câu hỏi đánh giá được xây dựng thủ công nhằm kiểm tra khả năng truy hồi và trả lời của hệ thống LegalRAG. Tập câu hỏi gồm 25 câu hỏi, được phân bố theo các lĩnh vực pháp lý tương ứng với tập văn bản, như trình bày trong Bảng 4.3.

Lĩnh vực	Số câu hỏi	Tỷ lệ
Hộ tịch	14	56%
Doanh nghiệp	3	12%
Nuôi con nuôi	5	20%
Bảo hiểm y tế	3	12%

Bảng 4.3. Phân bố câu hỏi test theo lĩnh vực

Các câu hỏi được thiết kế dựa trên nội dung thực tế của các văn bản pháp luật, tập trung vào các thông tin quan trọng như điều kiện thực hiện, hồ sơ cần thiết, lệ phí và thời hạn giải quyết thủ tục hành chính.

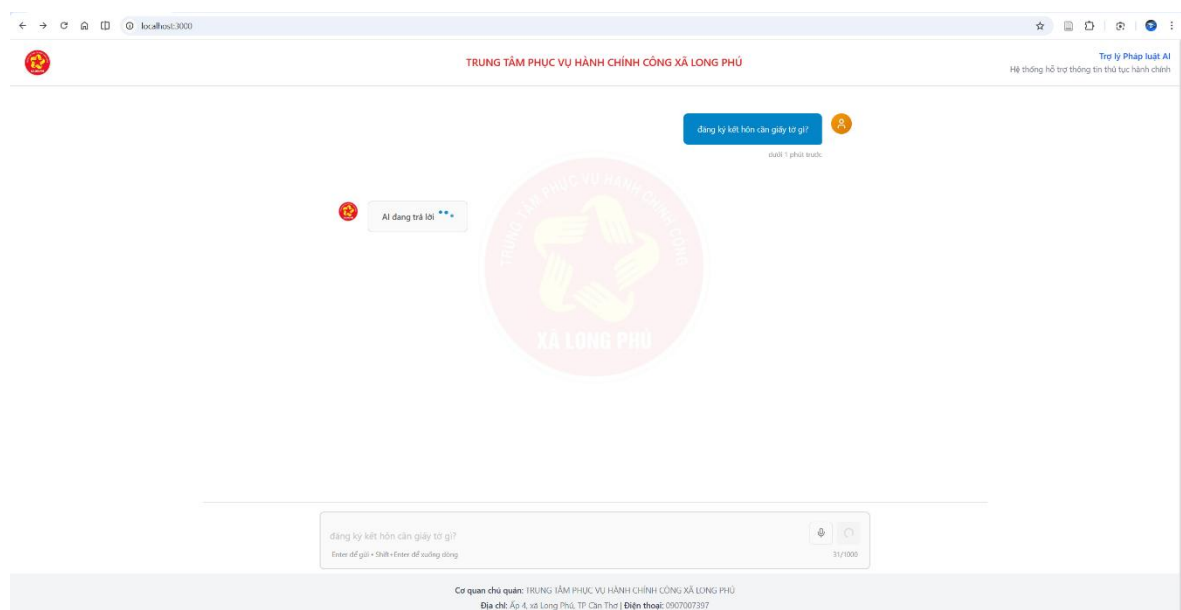
Mỗi câu hỏi trong tập đánh giá được gán nhãn với một văn bản pháp luật mong đợi (Expected Document), đóng vai trò làm ground truth trong quá trình đánh giá truy hồi ở mức văn bản. Ngoài ra, các đoạn nội dung liên quan (Relevant Chunk IDs) chứa trực tiếp thông tin trả lời cũng được xác định nhằm phục vụ cho việc đánh giá chất lượng truy hồi ở mức đoạn văn.

Việc xây dựng tập câu hỏi và gán nhãn được thực hiện thủ công nhằm đảm bảo độ chính xác và tính nhất quán của dữ liệu đánh giá.

4.2. Kết quả chức năng

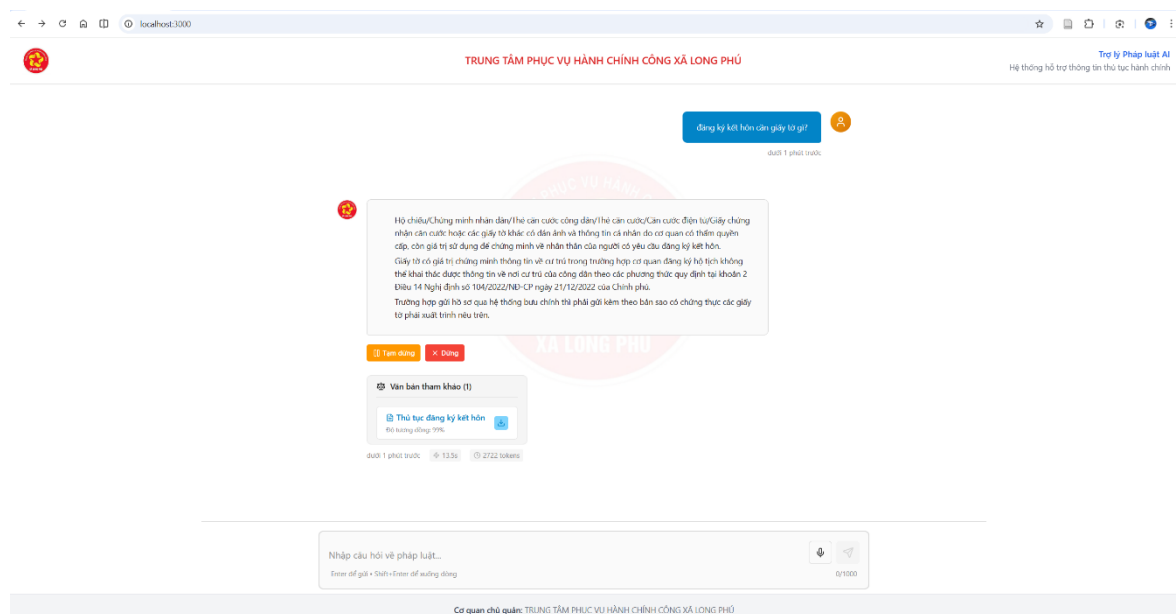
4.2.1. Chức năng hỏi đáp pháp luật

Hệ thống LegalRAG cho phép người dùng tra cứu thông tin thủ tục hành chính thông qua giao diện trò chuyện bằng ngôn ngữ tự nhiên tiếng Việt. Khi nhận được câu hỏi, hệ thống thực hiện truy hồi các văn bản pháp luật liên quan bằng phương pháp tìm kiếm vector kết hợp với tái xếp hạng kết quả nhằm xác định nguồn thông tin phù hợp nhất.



Hình 4.1. Giao diện nhập truy vấn hỏi đáp pháp luật

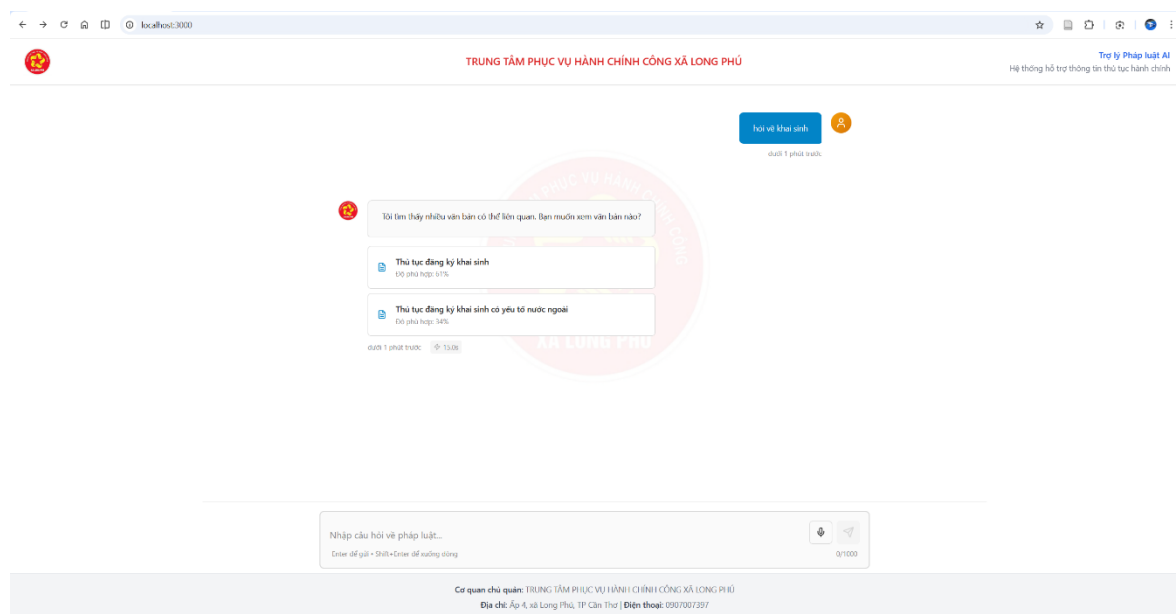
Sau khi xác định được các đoạn văn bản liên quan, mô hình ngôn ngữ lớn tổng hợp và sinh câu trả lời dựa trên nội dung văn bản gốc. Kết quả trả lời được hiển thị kèm theo trích dẫn nguồn nhằm đảm bảo khả năng kiểm chứng thông tin.



Hình 4.2. Kết quả trả lời và văn bản tham chiếu

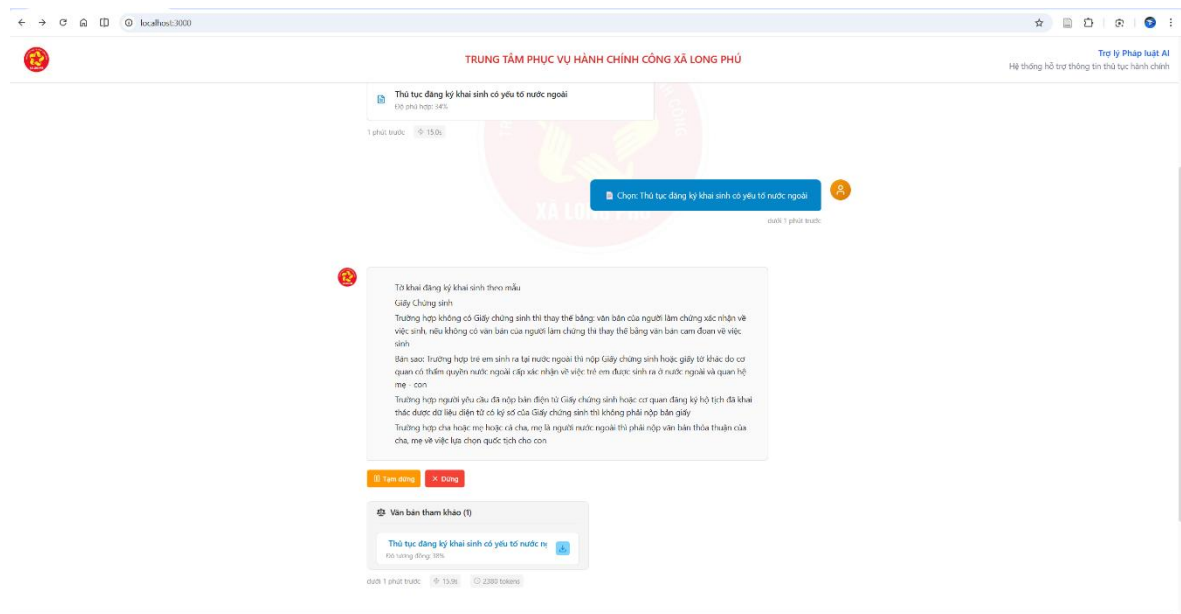
4.2.2. Cơ chế Clarification (làm rõ truy vấn)

Trong trường hợp câu hỏi của người dùng mang tính khái quát hoặc có thể liên quan đến nhiều thủ tục hành chính khác nhau, hệ thống LegalRAG không sinh câu trả lời ngay mà kích hoạt cơ chế Clarification để yêu cầu người dùng lựa chọn hoặc làm rõ phạm vi truy vấn.



Hình 4.3. Cơ chế yêu cầu làm rõ truy vấn

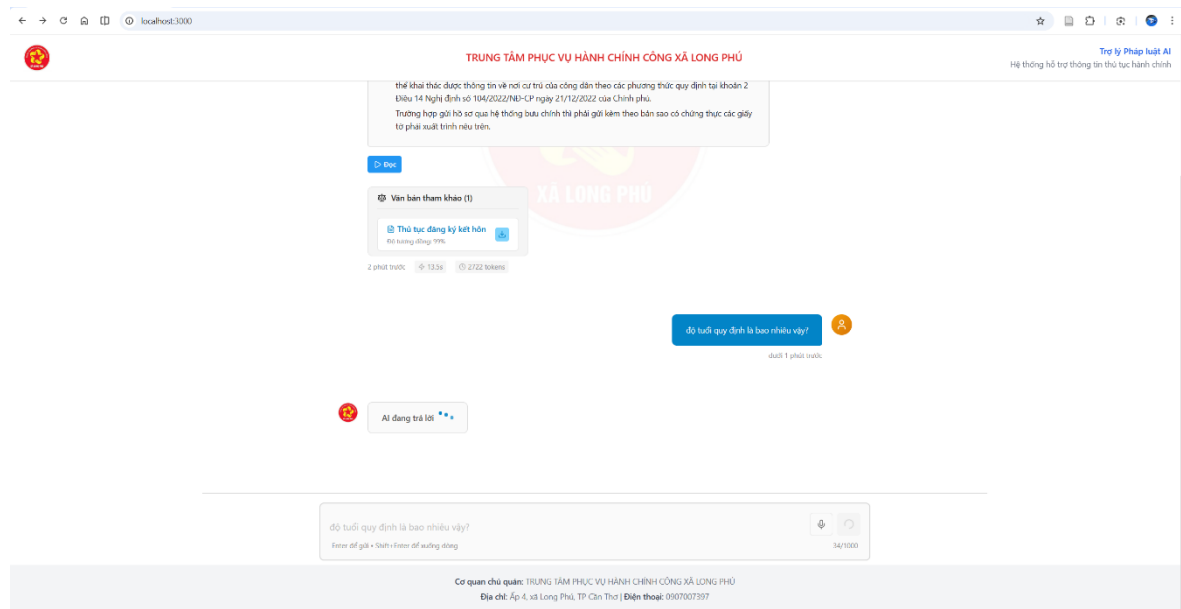
Sau khi người dùng xác nhận văn bản hoặc thủ tục cần tra cứu, hệ thống giới hạn phạm vi truy hồi trong văn bản đã được chọn và tiếp tục sinh câu trả lời chi tiết dựa trên nội dung tương ứng.



Hình 4.4. Kết quả trả lời sau khi làm rõ truy vấn

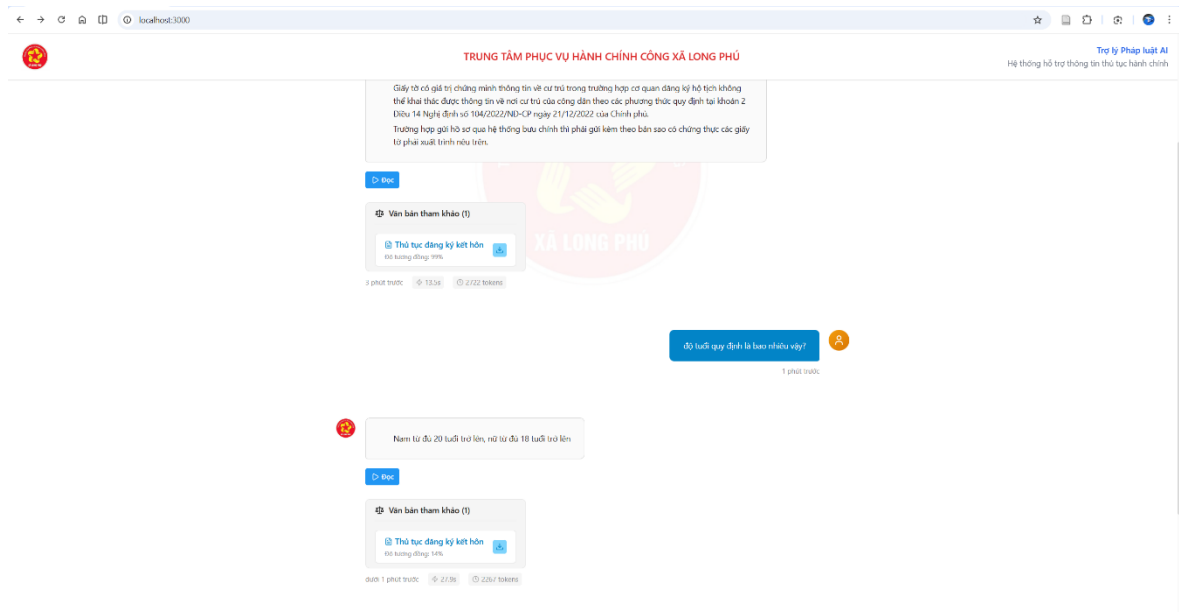
4.2.3. Conversational RAG (Document Pinning)

Trong quá trình hội thoại, khi hệ thống xác định được một văn bản pháp luật có độ liên quan cao và đủ độ tin cậy để sinh câu trả lời, cơ chế Document Pinning được kích hoạt. Theo đó, văn bản này được tự động “ghim” làm ngữ cảnh ưu tiên cho các lượt truy vấn tiếp theo, bao gồm cả các trường hợp truy vấn được làm rõ thông qua cơ chế Clarification.



Hình 4.5. Lựa chọn và ghim văn bản pháp luật

Sau khi văn bản được ghim, các truy vấn tiếp theo chỉ được truy hồi trong phạm vi văn bản đã được xác định, giúp hệ thống duy trì mạch hội thoại, giảm nhiễu từ các văn bản không liên quan và tăng độ chính xác của câu trả lời.



Hình 4.6. Trả lời dựa trên văn bản đã ghim

4.3. Đánh giá chất lượng câu trả lời

Để kiểm chứng hiệu quả thực tế của hệ thống LegalRAG, nghiên cứu tiến hành đánh giá trên tập dữ liệu kiểm thử (Test Set) bao gồm 25 câu hỏi thực tế thuộc các lĩnh vực: Hộ tịch, Doanh nghiệp, Nuôi con nuôi và Bảo hiểm y tế. Quá trình đánh giá tập trung vào khả năng truy xuất thông tin và chất lượng phản hồi của hệ thống. Chi tiết tập câu hỏi đánh giá và nhãn ground truth được trình bày tại Phụ lục.

4.3.1. Chiến lược Document-centric Retrieval

Khác với các phương pháp RAG truyền thống thường truy xuất thông tin ở mức đoạn văn (chunk-level retrieval) và trả về các đoạn rời rạc, hệ thống LegalRAG áp dụng chiến lược Document-centric Retrieval. Giá trị $k=10$ được chọn dựa trên nghiên cứu của Izacard & Grave (2021) trong FiD, cho thấy k trong khoảng 10-20 đạt cân bằng tốt giữa chất lượng và hiệu năng. Đặc thù của văn bản pháp luật đòi hỏi thông tin phải được đặt trong ngữ cảnh đầy đủ của từng thủ tục hành chính, do đó chiến lược truy xuất được thiết kế theo quy trình sau:

1. Vector Search: Truy xuất top-10 chunks có độ tương đồng cao nhất từ toàn bộ cơ sở dữ liệu vector.

2. Reranking (Tái xếp hạng): Tính điểm trung bình cho từng văn bản dựa trên các chunks đã tìm thấy (nhóm theo document_id).

3. Xác định Document Winner: Chọn ra văn bản có điểm số xếp hạng cao nhất (Document Winner).

4. Lọc và Xây dựng ngữ cảnh: Thay vì chỉ lấy 1-2 chunks khớp nhất, hệ thống thu thập các chunks liên quan thuộc về Document Winner và sử dụng top-5 chunks tốt nhất để xây dựng ngữ cảnh (Context) cho mô hình ngôn ngữ lớn (LLM).

Chiến lược này đảm bảo câu trả lời được tổng hợp từ một nguồn văn bản pháp luật thống nhất, hạn chế hiện tượng trộn lẫn thông tin giữa các thủ tục khác nhau, từ đó nâng cao tính nhất quán và độ chính xác pháp lý của câu trả lời.

4.3.2. Các chỉ số đánh giá

Nghiên cứu sử dụng hai nhóm chỉ số chính để đo lường hiệu năng:

1. Đánh giá ở mức đoạn văn (Chunk-level Retrieval):

- Precision@K: Tỷ lệ số đoạn văn trả về trùng khớp với ground truth trong top K kết quả.

- Recall@K: Tỷ lệ số đoạn văn ground truth được hệ thống tìm thấy trong top K kết quả.

- F1-Score: Trung bình điều hòa giữa Precision và Recall.

2. Đánh giá mức Document (Độ chính xác định danh văn bản):

- Hit Rate: Tỷ lệ câu hỏi xác định đúng văn bản pháp luật chứa câu trả lời.

- MRR (Mean Reciprocal Rank): Giá trị nghịch đảo trung bình của vị trí xuất hiện đầu tiên của văn bản đúng.

- Top-1 Accuracy: Tỷ lệ câu hỏi có văn bản đúng ở vị trí đầu tiên trong danh sách kết quả.

4.3.3. Kết quả đánh giá ở mức đoạn văn

Kết quả đánh giá chất lượng ở mức đoạn văn được trình bày trong Bảng 4.4.

Metric	Mean	Std	Min	Max	Mục tiêu tham chiếu	Đạt
Precision@5	0.260	±0.123	0.000	0.500	> 0.60	Chưa đạt
Recall@5	0.920	±0.277	0.000	1.000	> 0.60	Đạt
F1-Score	0.398	±0.162	0.000	0.667	> 0.60	Chưa đạt

Bảng 4.4. Kết quả đánh giá ở mức đoạn văn

Kết quả cho thấy Recall@5 đạt giá trị cao (0.92), trong khi Precision@5 và F1-Score ở mức thấp so với ngưỡng mục tiêu. Hiện tượng này không phản ánh chất lượng truy hồi kém của hệ thống, mà xuất phát từ sự khác biệt giữa cách gán nhãn ground truth và chiến lược Document-centric Retrieval được áp dụng.

Trong tập kiểm thử, mỗi câu hỏi thường chỉ được gán từ 1–2 đoạn văn chứa thông tin trả lời trực tiếp. Ngược lại, hệ thống LegalRAG có xu hướng truy hồi 3–5 đoạn văn thuộc cùng một văn bản pháp luật nhằm cung cấp đầy đủ ngữ cảnh cho mô hình ngôn ngữ lớn khi sinh câu trả lời.

Ví dụ, với câu hỏi “Thời hạn đăng ký khai sinh là bao lâu?”, ground truth chỉ đánh dấu một đoạn văn chứa thông tin “60 ngày”. Tuy nhiên, hệ thống trả về ba đoạn văn thuộc văn bản “Thủ tục đăng ký khai sinh”, trong đó chỉ một đoạn trùng khớp với ground truth. Trường hợp này dẫn đến Precision@3 bằng $1/3 \approx 0.33$, mặc dù câu trả lời sinh ra vẫn hoàn toàn chính xác. Các đoạn văn bổ sung tuy không nằm trong ground truth nhưng vẫn chứa thông tin hữu ích về thủ tục, hồ sơ, lệ phí - giúp LLM sinh câu trả lời toàn diện hơn.

Do đó, trong bối cảnh chiến lược Document-centric Retrieval, Recall@K và các chỉ số đánh giá ở mức văn bản có ý nghĩa phản ánh hiệu quả hệ thống tốt hơn so với Precision@K ở mức đoạn văn.

4.3.4. Kết quả đánh giá ở mức văn bản

Kết quả đánh giá khả năng xác định đúng văn bản pháp luật chứa câu trả lời được trình bày trong Bảng 4.5.

Metric	Giá trị	Chi tiết	Mục tiêu tham chiếu	Đạt
Hit Rate	92.0%	23/25	> 80%	Đạt
MRR	0.920	±0.277	> 0.70	Đạt
Top-1 Accuracy	92.0%	23/25	> 70%	Đạt

Bảng 4.5. Kết quả đánh giá ở mức văn bản

Các kết quả cho thấy hệ thống LegalRAG đạt độ chính xác cao trong việc xác định đúng văn bản pháp luật liên quan. Tỷ lệ Hit Rate đạt 92%, chứng tỏ trong phần lớn các trường hợp, hệ thống truy hồi đúng văn bản chứa thông tin trả lời.

Giá trị $MRR = 0.92$ cho thấy văn bản đúng thường xuất hiện ở vị trí đầu tiên hoặc rất sớm trong danh sách kết quả sau bước reranking. Kết quả này phù hợp với mục tiêu của chiến lược Document-centric Retrieval, trong đó việc xác định đúng văn bản pháp luật đóng vai trò quan trọng hơn so với việc truy hồi chính xác từng đoạn văn riêng lẻ.

4.3.5. Đánh giá chất lượng câu trả lời

Bên cạnh chất lượng truy hồi, nghiên cứu tiến hành đánh giá khả năng sinh câu trả lời của hệ thống thông qua các chỉ số phản ánh hành vi phản hồi thực tế, được trình bày trong Bảng 4.6.

Metric	Giá trị	Chi tiết	Ghi chú
Answer Rate	92.0%	23/25	Trả lời trực tiếp
Clarification Rate	8.0%	2/25	Yêu cầu làm rõ

Bảng 4.6. Chất lượng câu trả lời của hệ thống

Kết quả cho thấy hệ thống có khả năng trả lời trực tiếp phần lớn các câu hỏi trong tập kiểm thử. Tỷ lệ Clarification thấp (8%) phản ánh việc hệ thống chỉ yêu cầu làm rõ trong những trường hợp câu hỏi mơ hồ hoặc có thể liên quan đến nhiều thủ tục khác nhau.

Cơ chế Clarification được xem là một lựa chọn thiết kế có chủ đích, nhằm hạn chế sinh câu trả lời thiếu chính xác hoặc sai ngữ cảnh trong lĩnh vực pháp luật, thay vì cố gắng trả lời trong điều kiện thông tin chưa đủ rõ ràng. Trong lĩnh vực pháp luật, việc đưa ra câu trả lời sai có thể dẫn đến hậu quả nghiêm trọng hơn so với việc yêu cầu làm rõ. Do đó, Clarification Rate = 8% được đánh giá là hợp lý và phản ánh tính thận trọng của hệ thống.

4.3.6. Phân tích kết quả theo lĩnh vực

Để đánh giá hiệu quả của hệ thống trên từng nhóm thủ tục hành chính, nghiên cứu tiến hành phân tích kết quả theo lĩnh vực, như trình bày trong Bảng 4.7.

Lĩnh vực	Hit Rate	Số câu hỏi	Thời gian phản hồi TB
Hộ tịch	92.9%	14	2,730 ms
Doanh nghiệp	66.7%	3	2,817 ms
Nuôi con nuôi	100.0%	5	3,414 ms
Bảo hiểm y tế	100.0%	3	3,173 ms

Bảng 4.7. Kết quả đánh giá theo lĩnh vực

Kết quả cho thấy hệ thống đạt hiệu quả cao đối với các lĩnh vực có phạm vi thủ tục rõ ràng và nội dung văn bản tập trung như Nuôi con nuôi và Bảo hiểm y tế. Trong khi đó, lĩnh vực Doanh nghiệp có Hit Rate thấp hơn do số lượng câu hỏi và văn bản đánh giá còn hạn chế, dẫn đến độ bao phủ dữ liệu chưa cao.

Thời gian phản hồi trung bình giữa các lĩnh vực không có sự chênh lệch đáng kể, cho thấy hiệu năng hệ thống tương đối ổn định trong các kịch bản tra cứu văn bản pháp luật khác nhau.

4.4. Đánh giá hiệu năng

4.4.1. Thời gian phản hồi

Thời gian phản hồi được đo từ thời điểm hệ thống nhận truy vấn của người dùng đến khi câu trả lời hoàn chỉnh được trả về giao diện. Mỗi truy vấn được thực hiện một lần trong điều kiện hệ thống ổn định, không có tải đồng thời. Kết quả đo được tổng hợp và trình bày trong Bảng 4.8.

Metric	Giá trị	Mục tiêu	Đạt
Thời gian phản hồi trung bình	2,930 ms	< 15,000 ms	Đạt
Độ lệch chuẩn	±681 ms	—	—
Thời gian nhỏ nhất	1,453 ms	—	—
Thời gian lớn nhất	4,116 ms	< 15,000 ms	Đạt
P95 Latency	4,082 ms	—	—

Bảng 4.8. Kết quả đo thời gian phản hồi của hệ thống

Kết quả cho thấy hệ thống LegalRAG đạt thời gian phản hồi trung bình khoảng 2,9 giây, thấp hơn đáng kể so với ngưỡng yêu cầu 15 giây đã đặt ra trong các yêu cầu phi chức năng. Ngay cả trong trường hợp xấu nhất, thời gian phản hồi tối đa vẫn nằm trong giới hạn cho phép, đảm bảo trải nghiệm người dùng ổn định và mượt mà trong quá trình tra cứu thủ tục hành chính.

4.4.2. Phân tích thời gian xử lý theo từng bước

Để hiểu rõ hơn nguyên nhân ảnh hưởng đến thời gian phản hồi tổng thể, nghiên cứu tiến hành phân tích thời gian xử lý của từng bước trong pipeline hỏi đáp. Kết quả được trình bày trong Bảng 4.9.

Bước xử lý	Thời gian trung bình	Tỷ lệ
Embedding truy vấn	139 ms	4.7%
Vector search	6 ms	0.2%
Reranking	1,783 ms	60.8%
LLM Generation	987 ms	33.7%
Other	16 ms	0.5%
Tổng cộng	2,930 ms	100%

Bảng 4.9. Phân tích thời gian xử lý theo từng bước

Phân tích cho thấy bước Reranking chiếm tỷ trọng thời gian lớn nhất (60,8%) do mô hình cross-encoder phải tính toán độ liên quan cho từng cặp truy vấn–văn bản. Bước LLM Generation chiếm 33,7% tổng thời gian, trong khi đó các bước Embedding và Vector Search có thời gian xử lý rất nhỏ.

Đặc biệt, thời gian tìm kiếm vector chỉ khoảng 6 ms, cho thấy hiệu quả của việc sử dụng chỉ mục HNSW trong pgvector. Kết quả này khẳng định rằng thành phần truy hồi không phải là nút thắt cổ chai của hệ thống, mà hiệu năng tổng thể chủ yếu phụ thuộc vào bước tái xếp hạng và sinh câu trả lời.

4.4.3. Đánh giá mức độ đáp ứng yêu cầu phi chức năng

So sánh với các yêu cầu phi chức năng đã đề ra, hệ thống LegalRAG đáp ứng đầy đủ các tiêu chí về hiệu năng trong điều kiện triển khai thử nghiệm. Mặc dù ngưỡng yêu cầu thời gian phản hồi được đặt ở mức dưới 15 giây nhằm đảm bảo tính ổn định trong nhiều điều kiện triển khai khác nhau, kết quả thực nghiệm cho thấy hệ thống đạt hiệu năng tốt hơn đáng kể với thời gian phản hồi trung bình khoảng 3 giây.

Điều này cho thấy kiến trúc microservices kết hợp với chiến lược Document-centric Retrieval và cơ chế tối ưu truy hồi đã giúp hệ thống đạt được hiệu năng phù hợp cho các ứng dụng hỏi đáp pháp luật trong thực tế.

CHƯƠNG 5 KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1. Kết luận

Trong phạm vi đề tài, nghiên cứu đã xây dựng hệ thống hỏi đáp tự động trong lĩnh vực pháp luật Việt Nam dựa trên kỹ thuật Retrieval-Augmented Generation (RAG), nhằm hỗ trợ tra cứu thông tin thủ tục hành chính bằng ngôn ngữ tự nhiên tiếng Việt. Hệ thống được thiết kế với mục tiêu hiểu đúng câu hỏi của người dùng, truy xuất chính xác văn bản pháp luật liên quan và tổng hợp câu trả lời có căn cứ, kèm theo trích dẫn nguồn rõ ràng.

Kết quả nghiên cứu cho thấy hệ thống đã được hiện thực hóa thành công với kiến trúc microservices hoàn chỉnh, bao gồm các thành phần mã hóa ngữ nghĩa, tìm kiếm tương tự, tái xếp hạng độ liên quan và sinh câu trả lời bằng mô hình ngôn ngữ lớn. Hệ thống hỗ trợ hội thoại nhiều lượt, cơ chế yêu cầu làm rõ khi câu hỏi mơ hồ và chức năng điền biểu mẫu tự động, phù hợp với đặc thù của các thủ tục hành chính.

Kết quả thực nghiệm cho thấy hệ thống đạt hiệu quả cao trong việc truy xuất đúng văn bản pháp luật chứa thông tin trả lời, với thời gian phản hồi phù hợp cho các ứng dụng tương tác người dùng. Chiến lược Document-centric Retrieval giúp đảm bảo ngữ cảnh đầy đủ và nhất quán cho câu trả lời, trong khi cơ chế Clarification góp phần hạn chế hiện tượng trả lời sai ngữ cảnh đối với các câu hỏi chưa rõ ràng.

Về mặt thực tiễn, hệ thống LegalRAG có tiềm năng ứng dụng tại các cơ quan hành chính cấp cơ sở, đặc biệt trong bối cảnh phục vụ người dân tra cứu nhanh thông tin thủ tục hành chính mà không yêu cầu đăng nhập. Giải pháp này góp phần giảm tải cho cán bộ công chức, đồng thời tạo điều kiện để người dân tiếp cận thông tin pháp luật một cách thuận tiện và minh bạch hơn. Ngoài ra, nghiên cứu cũng đóng góp một quy trình đánh giá hệ thống RAG tiếng Việt có thể được tham khảo và tái sử dụng cho các nghiên cứu tiếp theo.

5.2. Hướng phát triển

5.2.1. Mở rộng dữ liệu

Bộ dữ liệu hiện tại chỉ bao gồm thủ tục hành chính của một số lĩnh vực đại diện. Hướng phát triển tiếp theo là mở rộng phạm vi dữ liệu bao gồm toàn bộ thủ tục hành chính cấp xã, huyện, tỉnh từ Cổng Dịch vụ công Quốc gia thông qua việc thu

thập thủ công hoặc sử dụng công cụ crawl từ các nguồn công khai. Ngoài ra, có thể tích hợp thêm các văn bản quy phạm pháp luật (Luật, Nghị định, Thông tư) để hệ thống có khả năng trả lời các câu hỏi về cơ sở pháp lý.

5.2.2. Cải thiện mô hình

Nghiên cứu có thể cải thiện chất lượng retrieval bằng cách fine-tune mô hình embedding trên tập dữ liệu pháp luật Việt Nam. Đối với reranking, việc sử dụng mô hình cross-encoder được huấn luyện đặc thù cho miền pháp luật sẽ giúp phân biệt tốt hơn các văn bản có nội dung tương tự. Ngoài ra, có thể thử nghiệm các kỹ thuật như query decomposition hoặc multi-hop retrieval cho các câu hỏi phức tạp.

5.2.3. Triển khai thực tế

Hệ thống LegalRAG được thiết kế theo định hướng triển khai cục bộ tại các cơ quan hành chính cấp xã, hoạt động như một trợ lý tư vấn pháp luật tự động hỗ trợ người dân tra cứu thông tin thủ tục hành chính. Hệ thống được cài đặt và vận hành trên một máy tính đơn lẻ, không yêu cầu kết nối internet trong quá trình sử dụng, phù hợp với điều kiện hạ tầng tại nhiều địa phương.

Trong tương lai, hệ thống có thể được tối ưu để hoạt động trên phần cứng phổ thông hơn, đồng thời phát triển giao diện kiosk cảm ứng đặt tại khu vực tiếp dân. Việc đóng gói hệ thống thành bộ cài đặt hoàn chỉnh sẽ giúp dễ dàng nhân rộng mô hình triển khai tại các xã, phường khác nhau.

5.2.4. Tối ưu hiệu năng và vận hành

Kết quả đánh giá cho thấy bước tái xếp hạng (reranking) hiện chiếm phần lớn thời gian xử lý tổng thể. Hướng cải thiện trong tương lai là sử dụng các mô hình reranker đã được distilled với kích thước nhỏ hơn, có thể chạy hiệu quả trên CPU mà vẫn đảm bảo chất lượng đánh giá độ liên quan.

Ngoài ra, việc thay thế mô hình ngôn ngữ lớn sử dụng API bằng mô hình LLM chạy cục bộ sẽ giúp hệ thống hoạt động hoàn toàn offline, phù hợp với các điểm triển khai không có kết nối internet ổn định. Đối với các câu hỏi thường gặp, cơ chế lưu đệm (caching) kết quả trả lời có thể được áp dụng nhằm giảm thời gian phản hồi và tăng trải nghiệm người dùng.

DANH MỤC TÀI LIỆU THAM KHẢO

- [1] P. Lewis *et al.*, “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” Apr. 12, 2021, *arXiv*: arXiv:2005.11401. doi: 10.48550/arXiv.2005.11401.
- [2] K. Guu, K. Lee, Z. Tung, P. Pasupat, and M.-W. Chang, “REALM: Retrieval-Augmented Language Model Pre-Training,” Feb. 10, 2020, *arXiv*: arXiv:2002.08909. doi: 10.48550/arXiv.2002.08909.
- [3] G. Izacard and E. Grave, “Leveraging Passage Retrieval with Generative Models for Open Domain Question Answering,” Feb. 03, 2021, *arXiv*: arXiv:2007.01282. doi: 10.48550/arXiv.2007.01282.
- [4] T. Mikolov, K. Chen, G. Corrado, and J. Dean, “Efficient Estimation of Word Representations in Vector Space,” Sept. 07, 2013, *arXiv*: arXiv:1301.3781. doi: 10.48550/arXiv.1301.3781.
- [5] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding,” May 24, 2019, *arXiv*: arXiv:1810.04805. doi: 10.48550/arXiv.1810.04805.
- [6] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks,” Aug. 27, 2019, *arXiv*: arXiv:1908.10084. doi: 10.48550/arXiv.1908.10084.
- [7] D. Q. Nguyen and A. T. Nguyen, “PhoBERT: Pre-trained language models for Vietnamese,” Oct. 05, 2020, *arXiv*: arXiv:2003.00744. doi: 10.48550/arXiv.2003.00744.
- [8] “dangvantuan/vietnamese-document-embedding · Hugging Face.” Accessed: Dec. 22, 2025. [Online]. Available: <https://huggingface.co/dangvantuan/vietnamese-document-embedding>
- [9] N. Ailon and B. Chazelle, “The Fast Johnson–Lindenstrauss Transform and Approximate Nearest Neighbors,” *SIAM J. Comput.*, vol. 39, no. 1, pp. 302–322, Jan. 2009, doi: 10.1137/060673096.
- [10] J. Johnson, M. Douze, and H. Jégou, “Billion-scale similarity search with GPUs,” Feb. 28, 2017, *arXiv*: arXiv:1702.08734. doi: 10.48550/arXiv.1702.08734.
- [11] *pgvector/pgvector*. (Dec. 22, 2025). C. pgvector. Accessed: Dec. 22, 2025. [Online]. Available: <https://github.com/pgvector/pgvector>
- [12] R. Nogueira and K. Cho, “Passage Re-ranking with BERT,” Apr. 14, 2020, *arXiv*: arXiv:1901.04085. doi: 10.48550/arXiv.1901.04085.
- [13] T. B. Brown *et al.*, “Language Models are Few-Shot Learners,” July 22, 2020, *arXiv*: arXiv:2005.14165. doi: 10.48550/arXiv.2005.14165.
- [14] OpenAI *et al.*, “GPT-4 Technical Report,” Mar. 04, 2024, *arXiv*: arXiv:2303.08774. doi: 10.48550/arXiv.2303.08774.
- [15] S. Newman, *Building Microservices: Designing Fine-Grained Systems*. O’Reilly Media, Inc., 2021.
- [16] C. Manning, P. Raghavan, and H. Schuetze, “Introduction to Information Retrieval,” 2009.

PHỤ LỤC

Tập câu hỏi đánh giá và nhãn ground truth

Phụ lục này trình bày danh sách 25 câu hỏi kiểm thử được sử dụng để đánh giá hệ thống LegalRAG trong Chương 4. Mỗi câu hỏi được gán nhãn với:

- Expected Document: Văn bản pháp luật chứa câu trả lời đúng
- Chunk ID: Định danh UUID của đoạn văn chứa thông tin trả lời trực tiếp (hiển thị 8 ký tự đầu)

Bảng PL.1. Danh sách câu hỏi test - Lĩnh vực Hộ tịch (14 câu)

ID	Câu hỏi	Expected Document	Chunk ID
1	Thời hạn giải quyết đăng ký khai sinh là bao lâu?	Thủ tục đăng ký khai sinh	7fe2854d
2	Cha mẹ có trách nhiệm đăng ký khai sinh cho con trong thời hạn bao nhiêu ngày?	Thủ tục đăng ký khai sinh	968b1631
3	Thời hạn xác minh tình trạng hôn nhân khi đăng ký khai sinh là bao lâu?	Thủ tục đăng ký khai sinh	44220614
4	Trường hợp cha mẹ lựa chọn quốc tịch nước ngoài cho con cần giấy tờ gì?	ĐK khai sinh có YTNN	6377736f
5	Đối tượng nào được thực hiện thủ tục đăng ký khai sinh có YTNN?	ĐK khai sinh có YTNN	1a2513d3
6	Điều kiện độ tuổi để đăng ký kết hôn là gì?	Thủ tục đăng ký kết hôn	545ed893
7	Khi nộp hồ sơ đăng ký kết hôn có cần văn bản ủy quyền không?	Thủ tục đăng ký kết hôn	de0d9c3c
8	Thời hạn xác minh tình trạng hôn nhân khi đăng ký kết hôn?	Thủ tục đăng ký kết hôn	b5e4b8c0
9	Cơ quan nào thực hiện đăng ký khai tử?	Thủ tục đăng ký khai tử	d4afcd8a
10	Thời hạn đăng ký khai tử là bao nhiêu ngày?	Thủ tục đăng ký khai tử	edde4612

ID	Câu hỏi	Expected Document	Chunk ID
11	Những trường hợp nào được miễn lệ phí đăng ký khai tử?	Thủ tục đăng ký khai tử	29c63ce4
12	Đăng ký khai tử cho người chết đã lâu cần giấy tờ gì?	Thủ tục đăng ký khai tử	a2e18ff2
13	Đăng ký giám hộ nộp bản sao có cần xuất trình bản chính không?	Thủ tục đăng ký giám hộ	836b76b4
14	Tờ khai đăng ký giám hộ theo mẫu nào?	Thủ tục đăng ký giám hộ	3d10e386

Bảng PL.2. Danh sách câu hỏi test - Các lĩnh vực khác (11 câu)

ID	Lĩnh vực	Câu hỏi	Expected Document	Chunk ID
15	Doanh nghiệp	Lệ phí đăng ký thành lập hộ kinh doanh do cơ quan nào quyết định?	ĐK thành lập hộ KD	be86327c
16	Doanh nghiệp	Thành phần hồ sơ đăng ký thành lập hộ kinh doanh?	ĐK thành lập hộ KD	dc940c81
17	Doanh nghiệp	Trường hợp ủy quyền đăng ký hộ kinh doanh cần giấy tờ gì?	ĐK thành lập hộ KD	51064b29
18	Nuôi con nuôi	Trường hợp nào được miễn lệ phí đăng ký nuôi con nuôi?	ĐK nuôi con nuôi	ab81a924*
19	Nuôi con nuôi	Thời gian kiểm tra hồ sơ và lấy ý kiến là bao lâu?	ĐK nuôi con nuôi	d8818542*
20	Nuôi con nuôi	Những trường hợp nào không được nhận con nuôi?	ĐK nuôi con nuôi	ae699776
21	Nuôi con nuôi	Hồ sơ đăng ký nuôi con nuôi cần những giấy tờ gì?	ĐK nuôi con nuôi	865901ed
22	Nuôi con nuôi	Thời hạn xác minh hoàn cảnh gia đình người nhận?	ĐK nuôi con nuôi	18e9eb89

ID	Lĩnh vực	Câu hỏi	Expected Document	Chunk ID
23	Bảo hiểm y tế	Trẻ em dưới 6 tuổi khám BHYT cần xuất trình giấy tờ gì?	KCB bảo hiểm y tế	89c87ba6
24	Bảo hiểm y tế	Phiếu chuyển cơ sở KCB có giá trị bao nhiêu ngày?	KCB bảo hiểm y tế	6325622e
25	Bảo hiểm y tế	Người hiến bộ phận cơ thể chưa có thẻ BHYT cần giấy tờ gì?	KCB bảo hiểm y tế	9d4cb50f

Ghi chú:

- YTNN: Yếu tố nước ngoài
- KD: Kinh doanh
- KCB: Khám chữa bệnh
- (*): Câu hỏi có nhiều hơn 1 chunk liên quan